IMPERIAL

MEng Individual Project

Imperial College London

Department of Computing

Beyond Before-and-After: Process-Quality Evaluation of Refactoring Sessions

Author:
Ethan Hosier

Supervisor:
Dr Robert Chatley

Second Marker:
Dr Cristian Cadar

May 30, 2026

Abstract

Your abstract goes here

Contents

Chapter 1

Introduction

1.1 Background

Refactoring is a routine activity in software development in which developers restructure code to improve readability, maintainability, and evolvability while preserving externally observable behaviour; traditional definitions describe it as a sequence of small, semantics-preserving transformations [1, 2, 3], and subsequent surveys categorise these refactoring operations and the tools that support them [4]. In practice, refactoring is widely encouraged because poorly structured code becomes an obstacle when enhancing and maintaining code, and long-lived systems need continual adjustment (as argued in work building on Lehman's laws and empirical studies of software maintenance). At the same time, refactoring can be costly: it consumes developer time, introduces risk, and is frequently intertwined with other work such as feature development and bug fixing rather than occurring as isolated "refactoring-only" tasks [5, 6].

1.2 Motivation

Most research and tooling that evaluates refactoring focuses on *outcomes* rather than the *process*. A common approach is to measure refactoring success as a change in static quality indicators between a "before" and "after" state: reductions in code smells, improvements in cohesion/coupling, decreases in complexity, or related metrics to do with maintainability. Typical strands of work include metric-driven detection or prioritisation of candidate refactorings (e.g., using metric changes to separate refactoring-like edits from functional ones, or to choose between different refactoring options), as well as refactoring recommendation systems that rank alternatives by their impact on design metrics. More recent work expands the signals used for guidance by incorporating traceability alongside code metrics when ranking refactoring solutions, and exploring how product/process metrics relate to developers' motivations to refactor (e.g., studies building on mined refactorings and repository history, and proposals that use LLMs to determine developers' motivations from commit artefacts). Alongside this, there is a substantial line of work that has improved the ability to *detect* refactorings from code changes, notably by using AST-based differencing and structured matching to infer refactoring operations between revisions (e.g., RefactoringMiner and related approaches), which has enabled large-scale empirical studies of how often refactoring happens, what kinds occur, and in what contexts [7, 8].

However, these perspectives evaluate refactoring as a mapping from an initial program state to a final program state, leaving the *trajectory* between those states largely outside the picture. This matters because developers rarely refactor in a single atomic operation: they work through intermediate states, sometimes take detours, redo work, and often accept temporary degradation (e.g., in readability, modular structure, or test/build stability)

before arriving at a final outcome. Two developers can arrive at the same end state via very different paths: one might take lots of disruptive steps, bounce between designs, or create avoidable churn, while another might get there via a steadier, more stable sequence. Existing endpoint-only evaluation cannot distinguish these different cases. Meanwhile, research on process realism suggests that refactoring is often "flossed" into ongoing development instead of being a separate dedicated task, which increases the chance of mixed-intent steps and noisy traces [5]. This makes the quality of the process (how a developer navigates intermediate states) potentially important for both developer productivity and the health of the codebase as it evolves.

Recent work has started to look more closely at refactoring trajectories themselves. RefactorBench, for example, uses a state/action/observation view to study how language-model agents fail on multi-file refactoring tasks [9], while Bouzenia et al. characterise recurring action patterns in software-engineering agent traces [10]. A recent systematic literature review also surveys how LLM-based refactoring work is evaluated and highlights several open problems in this area [11]. However, this still leaves a gap for evaluating an observed developer trace directly. The closest related systems either synthesise a path to a desired end state, without judging the path the developer actually took, or recommend refactorings towards an architectural target, without reference to an observed process. This project addresses that gap by scoring recorded refactoring traces with an explicit process-quality metric, generating higher-scoring counterfactual reference trajectories, and identifying measurable points where the observed trace diverges from those references.

1.3 Research Objectives

This thesis asks whether, given the start and end state of a refactoring session, we can judge the quality of the path the developer took and point to places where a better path was available. The core idea is to model refactoring as a *process* over states, rather than judging it only by comparing endpoints. To do this, we adopt a state/action/trace framing: snapshots of the codebase are treated as *states*, and developer edits-including IDE refactorings and manual transformations-are treated as *actions* that produce a trace from the start state to the end state.

A core component will be a **process-quality metric** that scores traces according to criteria that will be defined and justified using existing research on code quality metrics, smell detection validity, and process-level signals. The metric is not assumed to be purely structural: it may incorporate multiple terms capturing endpoint improvement, intermediate-state degradation, churn/disruption, and "safety" proxies such as preserving build and test behaviour, reflecting the known limitations of relying on any one signal alone.

To go beyond scoring a single observed refactoring trace we also construct *counterfactual* alternatives: **reference trajectories** between the same start and end states that score better under the chosen process-quality metric. These references give the observed trace something concrete to be compared against. The analysis can then identify *divergence points*, where the developer's path moves away from a higher-scoring alternative, and estimate how much each divergence contributes to the final quality gap. This is related to systems that generate or recommend refactoring sequences: ReSynth synthesises a path to a target end state from a developer's partial edits [12], Refactoring Navigator recommends steps towards an architectural target [13], and search-based refactoring treats sequences as an optimisation problem under explicit quality functions [14, 15]. The difference is that these systems are mainly concerned with producing a useful sequence. Here, the generated trajectory is used as a reference for evaluating the process the developer actually followed.

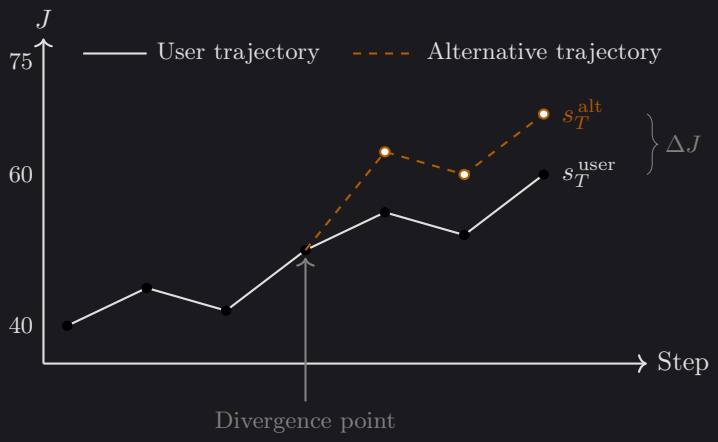

Figure 1.1: Illustration of two refactoring trajectories scored by the process-quality metric J. The solid line shows the developer's observed trajectory, while the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state. The gap ΔJ is the estimated contribution of that divergence to the final score difference.

1.4 Contributions

The main contributions of the project are:

- A process-quality metric for refactoring trajectories. The metric combines terms from prior work on code quality and developer process signals – including both endpoint cleanliness gain and an intermediate-cleanliness lag penalty so that two trajectories which finish at the same code can still score differently if one front-loaded the cleanup – and we test how much the results change under sensitivity and ablation experiments.
- A divergence-point detector that compares an observed trace with a higher-scoring reference trajectory. The detector groups divergences into four kinds: *Ordering*, *Manual-Refactor*, *Rework*, and *Hygiene*.
- A 45-session dataset of recorded refactoring sessions on a purpose-built Java codebase. We provide multi-label ground-truth annotations and the experimental scripts used for the sensitivity, ablation, and divergence analyses.
- An empirical evaluation of the detector, reporting precision and recall for each divergence kind. The evaluation also shows which parts of the score formula are useful in practice, and which kinds of process improvement it does not capture well.

1.5 Approach and Scope

The approach is intended for realistic refactoring sessions, where IDE-supported refactorings and manual edits are often mixed together. In this project we focus on Java, since it gives us mature parsing tools, established quality metrics, and refactoring mining support such as RefactoringMiner. We represent a refactoring session as a sequence of code snapshots. Each step may correspond to a burst of manual edits or the use of an automated IDE refactoring tool, and the same model could later be applied to traces at different levels of granularity where that data is available.

The work does not attempt to enumerate every possible refactoring sequence or prove that a generated trajectory is globally optimal. Instead, it uses the analysis component to find a plausible higher-scoring reference path between the same start and end states.

This keeps the task focused on explaining the developer's observed process, rather than replacing the developer with an automated refactoring planner.

The work is scoped to human developer sessions. The agent comparison in Chapter 4 is a feasibility extension on the same tooling rather than a co-equal evaluation target: the detectors are designed against human edit patterns (IDE refactor invocations, edit-burst structure, commit-cadence windows) and do not map cleanly onto agent traces. We treat that mismatch itself as one of the things Chapter 4 surfaces.

1.6 Thesis Roadmap

The remainder of the thesis is organised as follows. Chapter 2 reviews related work on refactoring evaluation, refactoring mining, and refactoring recommendation. Chapter 3 defines the process-quality metric and divergence-point detector. Chapter 4 describes the implementation of the tool. Chapter 5 presents the evaluation and discusses threats to validity. Chapter 6 concludes the thesis and outlines possible future work.

Chapter 2

Background

This chapter reviews the background needed to treat *refactoring as a process*, rather than only as a change between two endpoints. It begins with definitions of refactoring and refactoring sequences (§2.1), then discusses outcome-based evaluation and the code-quality signals commonly used to judge whether code has improved (§2.2–§2.3). It then reviews refactoring mining, recommendation and sequence synthesis, and IDE-based process tracing (§2.4–§2.6). The chapter ends by positioning this project against that work (§2.7).

2.1 Definitions and framing

2.1.1 Refactoring as behaviour-preserving transformation

Refactoring is usually defined as changing a program's internal structure to improve qualities such as readability or maintainability, while preserving externally observable behaviour. Classic definitions stress that refactoring is built from small, semantics-preserving steps, with safety supported by preconditions and validation such as testing [1, 2, 3]. Opdyke's work frames refactorings as program transformations with applicability conditions intended to preserve behaviour under a static approximation [1]. Fowler's catalogue helped popularise refactoring as an incremental developer activity made up of many small operations [2, 3]. Mens and Tourwé later survey the wider research landscape, bringing together definitions, taxonomies, and tool support, and emphasising the role of precondition checking and program analysis in automated refactoring [4].

In practice, this clean definition is harder to apply. Developers often refactor while also adding features or fixing bugs, and some refactorings are only partly completed. As a result, it is often ambiguous whether a change seen in version history is a "pure refactoring". In this thesis, behaviour preservation is therefore treated as an *intended property* of refactoring steps, while recognising that real traces may include edits with more than one purpose.

2.1.2 Atomic refactorings, smells, and sequences

Research on refactoring often describes refactorings as a set of *operators*, or atomic refactorings, such as Extract Method, Move Method, and Rename [2, 4]. Larger changes can then be understood as sequences of these smaller operations. Code smells are closely related: they are heuristic indicators of possible design problems, and therefore of possible refactoring opportunities [2, 3]. Although smells are not formal correctness criteria, they provide a useful bridge between developer judgement and measurable signals, discussed further in §2.3.

A key distinction in this thesis is between judging only the start and end states of a refactoring, and judging the path taken between them. Two developers may reach the same final code, but one path may involve more temporary degradation, churn, or disruption

than the other. These differences can matter for productivity and risk, even when the endpoint looks the same.

2.1.3 Representing refactoring as states and actions

To represent this refactoring path explicitly, we use a state/action/trace model. A codebase snapshot is treated as a *state* $s \in \mathcal{S}$, such as the repository at a particular point in time. An *action* $a \in \mathcal{A}$ is a developer edit step, which may be an IDE-supported refactoring, a manual change, or a mixture of both. A refactoring process is then a *trace*

$$\tau = (s_0, a_0, s_1, a_1, \ldots, a_{T-1}, s_T),$$

where each action a_t transforms state s_t into s_{t+1}. This makes it possible to talk about "process quality" as a property of the whole trace τ, rather than only as a comparison between the first and final states (s_0, s_T).

This framing is also close to recent software-engineering work that models agent and developer behaviour as a partially-observable Markov decision process, where a trajectory is a sequence of action and observation pairs [9]. We keep the explicit state notation because this thesis scores the intermediate code states themselves, not only the actions or observations produced along the way.

2.2 Outcome-based refactoring evaluation

2.2.1 Judging refactoring by the final state

Refactoring is often evaluated by comparing quality indicators before and after the change. Under this view, a refactoring is successful if the final snapshot improves on the starting snapshot according to selected measures, such as lower coupling, higher cohesion, lower complexity, or fewer code smells. This endpoint-focused view appears both in empirical studies of refactoring impact and in recommendation systems that rank candidate refactorings by their predicted improvement.

A clear example is *search-based refactoring*, where refactoring is treated as an optimisation problem over possible refactoring operations or sequences. Harman and Tratt apply multi-objective search at the design level, treating coupling and cohesion as objectives that may need to be traded off against each other [14]. Later work extends this idea to many-objective settings and adds constraints or preferences for practical concerns such as test coverage or prioritisation [15]. This line of work is useful here because it shows that "better" code is not a single fixed property: different quality goals can pull in different directions.

The same endpoint-based idea is also used in refactoring recommendation systems. These systems compare possible refactorings and rank them by the improvement they are expected to produce. Recent work has looked at how to choose among large sets of candidate refactorings, and which code characteristics are associated with positive or negative effects in practice [16].

2.2.2 What endpoint evaluation captures-and what it misses

Endpoint-based evaluation is useful when the main concern is whether the final code is better than the starting code under some metric Q. It gives recommendation systems a simple strategy: simulate each candidate refactoring and choose the one that gives the largest improvement in Q. It is also useful for empirical studies, where refactoring events can be compared with changes in measured code quality. This fits naturally with many existing metrics, since they are defined over a single codebase snapshot.

The weakness of this approach is that it says little about the refactoring process itself. Two developers may reach the same endpoint through very different paths, but endpoint metrics would treat those paths as equivalent. In practice, the intermediate states can still matter: code may become harder to understand for a period of time, the modular structure may be disrupted, or the build and tests may break before being repaired. The path can also be costly in other ways, for example by creating extra churn, making review harder, or increasing integration risk. Empirical work on modularity improvement shows this trade-off clearly: improvements in design metrics can come with substantial structural disruption [17]. This motivates evaluating not only the final code, but also the stability and cost of the path taken to reach it.

This is the point at which the process view becomes useful. Final-code metrics still matter, but they need to be considered alongside what happened on the way there: whether the work introduced temporary degradation, required rework, created unnecessary churn, or broke build/test stability.

2.3 Code smell detection and quality metrics

2.3.1 Using code smells as design signals

Code smells provide a way to connect qualitative design concerns with measurable indicators [2]. Smell detectors usually identify these cases either through hand-written rules or learned models. They often rely on structural properties such as size, coupling, and cohesion, with some tools also using lexical or semantic features. One prominent rule-based example is DECOR, which defines smells and design anti-patterns through measurable symptoms and detection rules [18]. DECOR is widely used in empirical studies because its definitions are explicit and it can be run efficiently at scale.

Smell detectors are useful, but they should not be treated as ground truth. Comparative studies show that tools can disagree in both precision and recall, and that performance varies across systems and smell types [19]. Much of this comes from the practical choices each tool makes, such as thresholds, feature sets, and implementation details. Detectors can also produce false positives, marking code as "smelly" even when there is no clear negative impact or when the design reflects an acceptable trade-off [20]. For this reason, smell counts are used in this thesis as one signal among several. They are combined with structural metrics, readability-oriented measures, and process-level safety signals, rather than being allowed to determine the process-quality score on their own.

2.3.2 Structural and readability metrics

Static metrics provide another way to measure code quality at the level of classes, methods, and packages. Common metric families cover coupling, cohesion, complexity, and size, and are often used as indicators of maintainability. These structural measures are useful, but they do not capture everything developers care about when refactoring. Readability is one example: models that combine structural features, such as line structure, with textual features, such as identifier patterns, tend to align better with human judgements of readability [21]. This matters for process evaluation because readability and understandability are common reasons for refactoring, and they can change noticeably in intermediate states even if the final code improves.

2.3.3 What this means for the process-quality metric

The process-quality metric $J(\tau)$ uses these signals, but it also has to account for their limitations. The literature suggests three design choices:

1. **Use multiple signals.** No single metric or smell detector gives reliable ground truth across all contexts [19, 20]. Combining several signals reduces dependence on the quirks of one tool.

2. **Separate endpoint improvement from process cost.** Improving the final snapshot is important, but disruption and temporary degradation can still matter even when the endpoint improves [17].

3. **Keep the components interpretable.** A process critique is more useful when it can point to a specific issue, such as a readability dip, churn spike, or smell increase, rather than only reporting a single opaque score.

A process metric should also include some notion of developer **effort**, such as the number of steps in the trajectory or the size of the edits between successive states. Fewer steps are not always better, since small steps can reduce risk, but step count and edit size are still useful for identifying unnecessary detours and rework. Similar cost terms are already used alongside structural quality goals in search-based refactoring and recommendation work, for example when trying to improve quality while minimising change [14, 15].

Table 2.1 summarises the candidate signals, their main benefits and limitations, and examples of Java tooling. The table does not define the final formula for $J(\tau)$, including weights or normalisation. Its purpose is to motivate the signals used later; the precise definition of $J(\tau)$ is given in the methodology chapter and used consistently in the evaluation.

Signal	Pros	Cons / threats	Typical tooling
Structural metrics (coupling/cohesion/ complexity/size)	Widely used; efficient; interpretable at class/method granularity	Validity depends on context; may incentivise disruptive restructuring; metric interactions	Static analysis metric extractors; IDE/CI integrations
Smell counts / severity (rule-based)	Directly targets design issues; aligns with developer's motivations	Detector disagreement and false positives; threshold sensitivity; context dependence	DECOR-style detectors [18]; other smell tools [19]
Readability models	Closer to human comprehension than structural-only metrics; captures lexical/style clues	Model generalisation and dataset bias; may be language/style dependent	Readability predictors [21]
Churn / disruption (LOC changed, file moves/renames)	Captures cost and instability of change; naturally trajectory-oriented	Churn may be necessary for large refactors; can penalise required restructuring	VCS mining; diff statistics
Process length / effort (number of steps; edit magnitude)	Trajectory-native cost signal; discourages detours/rework; easy to compute	Sensitive to step granularity (commits vs. checkpoints vs. IDE events); may penalise safe micro-steps; requires calibration	VCS checkpoints; IDE logs; diff/AST edit size
Build/test preservation signals	Direct safety signal; aligns with "behaviour-preserving intent"	Requires runnable builds/tests; noisy due to flaky tests; environment dependence	CI logs; local build/test runs

Table 2.1: Candidate signals for constructing a process-quality objective. We combine endpoint quality change with trajectory-oriented cost and stability terms, rather than relying on any single indicator.

2.4 Refactoring detection and mining from code changes

2.4.1 From diffs to refactoring actions

To evaluate a process trace in practice, developer actions either need to be captured directly, for example through IDE event logs, or inferred from the differences between snapshots such as bursts of manual user edits. Much of this work uses the second route: given two versions of a program, infer whether a refactoring happened and identify which refactoring types occurred.

Structured code differencing is important here because many refactorings are not easy to recognise from line-based diffs alone. AST-based differencing can represent moves, renames, and other non-local edits more directly. GumTree, for example, maps fine-grained AST nodes between program versions and produces edit scripts that better reflect the structural changes developers intended [8]. This kind of differencing is often used as the base layer for refactoring detection.

Modern refactoring miners use these structural mappings together with heuristics for particular refactoring types. RefactoringMiner detects a wide range of refactorings between two versions of the code by matching related classes, methods, and fields [7]. RefDiff takes a similar approach, using similarity heuristics and static analysis to detect common refactoring types in version histories [22]. For process analysis, tools like these are useful because they can turn a transition $s_t \rightarrow s_{t+1}$ into a set of detected refactoring actions, giving the trace a concrete *action vocabulary*.

2.4.2 What refactoring detectors miss

Refactoring detection tools usually report what kind of refactoring occurred, such as Move Method or Extract Class, and which classes, methods, or fields were involved. This fits the operator-based view of refactoring used in refactoring guides and surveys [2, 4]. However, the literature also reports several limitations that matter for process evaluation.

Mixed and tangled changes. Version control commits often mix refactoring with feature work or bug fixes, and they may bundle several unrelated activities together. A single commit-level transition can therefore include both behaviour-preserving and behaviour-changing edits. In these cases, refactoring detection may return only partial or ambiguous results. This makes snapshot-derived actions harder to interpret and motivates careful trace construction, such as using finer checkpoints and methods that can tolerate imperfect refactoring detections.

Limited coverage and combined refactorings. Detectors only cover a finite set of operators, and refactorings can be composed, such as a move followed by a rename, in ways that make matching harder. Some refactorings are also performed manually without tool support and may not fit the patterns that detectors expect. For process evaluation, this means that "actions" should be treated as partially observed: detected refactorings are useful features, but they are not a complete description of developer intent.

Behaviour preservation is not verified. Detection is usually structural. It infers that a change *looks like* a refactoring, but it does not prove that behaviour was preserved. Because of this, downstream evaluation should not treat a detected refactoring as automatically safe. Where possible, it should also use independent safety signals, such as whether the build and tests still pass.

2.4.3 Tie-in to our work

Refactoring mining is important infrastructure for building traces and describing actions at scale [7, 22]. This thesis does not try to improve refactoring detection accuracy. Instead, mined refactorings are used as part of the observational layer: they provide labels and features for transitions between states, which can then support process scoring and reference-trajectory generation. The next section builds on this by discussing work that generates and navigates refactoring sequences.

2.5 Refactoring recommendation and search/synthesis

These systems generate paths; we evaluate observed ones. The literature on refactoring recommendation, sequence synthesis, and goal-directed navigation is therefore mainly relevant as a contrast: it establishes that refactoring can be treated as a structured sequence under explicit objectives, but the generated sequence is the object of interest rather than a developer's actual trace. This section gives the contrast in one subsection; the more recent trajectory-formalism work that does engage with observed traces, which is closer to our framing, gets its own subsection.

2.5.1 Recommendation and synthesis as a contrast

Several strands of prior work show that refactoring sequences can be generated and optimised under explicit objectives. ReSynth, for example, synthesises a sequence of IDE-supported refactorings consistent with a developer's partial edits, using the touched classes, methods, and fields together with operator preconditions to constrain the search [12]. Its goal is reachability rather than process quality, but it shows that a developer trace can provide useful evidence for constructing a refactoring path. Search-based refactoring takes a different route: Harman and Tratt frame refactoring as a multi-objective optimisation problem over design qualities such as cohesion and coupling, producing Pareto fronts rather than a single best answer [14]. Later many-objective work adds concerns closer to everyday development, including the number of refactorings performed [15] and architectural distance as an explicit disruption objective [23]. Refactoring Navigator is closer to a developer-facing recommender: it starts from a current system and a desired architecture, then recommends a sequence of refactoring steps using a search-based strategy [13]. Related empirical work on modularity optimisation also shows that improving structural metrics can substantially disturb the original modular structure [17], which motivates the disruption-aware penalties used later in §3.

This thesis builds on the idea that refactoring sequences can be generated under explicit objectives, and the quality terms reported by Harman and Tratt, Mohan and Greer, Paixão et al., and Cortellessa et al. inform the weights of the process-quality score (§3.2.2). The difference is in what the generated sequence is used for. In recommendation and navigation systems, the generated sequence is usually the output: it is the path the developer is meant to follow. In this thesis, the generated trajectory is a counterfactual reference. The developer's observed trace is the object being evaluated, and the synthesised alternative exists only to give that trace something concrete to compare against. Existing recommendation and navigation systems do not judge the intermediate choices the developer actually made, nor explain where the observed path moved away from a better alternative [12, 13]. That is the gap this thesis addresses.

2.5.2 Recent trajectory-formalism work in software engineering

Recent software-engineering work has also begun to describe development in terms of trajectories. RefactorBench, for example, represents agent behaviour as a sequence of actions and observations in a POMDP-shaped environment [9]. Other work looks for recurring action patterns in agent traces [10] or builds training environments around software-engineering tasks [24]. Alomar et al. similarly note that evaluation remains an open issue in LLM-based refactoring research [11]. These papers are useful because they treat the path through a task as something worth analysing, not just the final answer. However, their focus is usually on language-model agents and benchmark success, rather than on the quality of a human developer's refactoring process. Observational work on real refactoring practice is closer to the setting of this thesis: it shows that refactoring normally unfolds as an ordered sequence of operations, often mixed with feature work, fixes, or cleanup [25]. This thesis adopts the human-developer scope. Agent sessions appear in Chapter 4 as a feasibility extension applied to the same scoring formula and detector design, not as a second contribution.

2.6 Process-aware work and IDE trace capture

Evaluating refactoring as a process requires more than the before and after versions of the code. It needs a trace: a record of the intermediate states and the actions that produced them. Version-control histories provide one source of this information, but commits and checkpoints are often too coarse. IDE-level logs can capture a finer view of what the developer actually did, including edits, navigation, test runs, and refactoring commands. This section therefore focuses on process-aware logging work, both because it shows that useful traces can be collected and because it exposes the practical difficulties of interpreting them.

2.6.1 Logging developer interactions in IDEs

Several studies have used developer-interaction logs to recover workflows and common behaviour patterns. Damevski et al., for example, mine sequences of Visual Studio interactions from telemetry-like event streams, aiming to find frequent usage patterns without recording the full source code [26]. This is useful evidence that commands, navigation actions, and edits can be captured in a structured form while still protecting developer privacy. The limitation is that such logs do not preserve enough project state to replay a session later, which restricts the kind of process-quality analysis they can support.

Poncin et al. take a broader repository-level view, combining version-control, bug-tracker, and mail-archive data to infer developer workflows using process mining techniques [27]. Their work shows how low-level records can be turned into higher-level activity models, such as sequences of commits, issue updates, and discussions. It also illustrates a recurring problem: development traces are noisy. Developers may work on several tasks at once, and raw events have to be segmented before they can be treated as meaningful steps. Repository-level traces also miss IDE-specific information, such as which command was run or whether a refactoring was invoked through the IDE.

More recent logging infrastructure has often focused on developer–LLM interaction rather than refactoring-specific traces. *CodeWatcher*, for example, is a lightweight VS Code extension that records fine-grained IDE events such as LLM-tool insertions, deletions, copy-paste actions, and focus changes, with timestamps that support later reconstruction of a coding session [28]. The same style of event capture would be useful for refactoring, but the setting does not map directly onto this thesis: the work is centred on VS Code and LLM-assisted coding, whereas this project studies Java refactoring sessions in IntelliJ.

Recent trace-capture work is therefore relevant in principle, but not directly reusable for the setting studied here.

Taken together, existing datasets do not provide exactly the trace needed for this thesis. Some capture rich IDE events, but for a different activity; others capture refactoring-related activity, but only at repository granularity; and others do not retain enough code content to replay the trajectory. The dataset used here therefore has to be built for the task rather than reused directly from prior work. Chapter 3 returns to this in §3.7.

2.6.2 Granularity, segmentation, and semantic interpretation

IDE logging also raises a modelling question: what counts as one "step" in a refactoring process? Commits are usually too coarse, since they often mix refactoring with feature work, fixes, or cleanup. Raw IDE events are too fine-grained: navigation, edits, refactoring-tool calls, and test runs appear as separate events even when they belong to one conceptual change. The process-mining and interaction-analysis literature shows that grouping this kind of activity into meaningful tasks or episodes is difficult [27, 26]. For refactoring, this matters because one refactoring may involve many small interactions, while a short burst of editing may contain several distinct actions.

A process-evaluation system therefore needs a step definition that is both reliable to extract and meaningful to interpret.

2.7 Summary and positioning: toward process-quality evaluation

The literature discussed in §2.1–§2.6 gives a strong starting point for this thesis, but it also shows where existing work stops short of process-level evaluation.

Refactoring itself is well established as behaviour-preserving change, supported by catalogues of operators and by tool-based precondition checking [1, 2, 4]. Most evaluation work, however, still judges refactoring by comparing start and end snapshots, using structural metrics, code-smell counts, or related quality indicators [14, 18]. These signals are useful, but they are imperfect and can disagree across tools, so combining several signals is often more reliable than relying on one measure alone [19, 20, 21]. Refactoring mining and AST differencing make it possible to infer actions from code changes and reconstruct traces from repository histories [7, 8]. Recommendation and synthesis systems show that refactoring sequences can be generated and planned towards final goal states under explicit quality objectives [12, 13, 14, 15], but treat the generated sequence rather than a developer's observed trace as the object of evaluation. IDE logging work adds that fine-grained traces can be captured, but also shows how difficult they can be to segment and interpret [27, 26, 29].

There has therefore been substantial work on *detecting* refactorings, *recommending* refactorings, and *synthesising* refactoring sequences. What is still missing is a system that scores the path a developer actually took against an explicit process-quality metric, then compares that path with higher-scoring alternatives. The closest existing work usually treats the generated sequence as the answer: if it reaches a good target state, the system has done its job. In this thesis, the observed developer trace is the object being evaluated. The aim is to judge whether the process itself was good, including whether it introduced unnecessary intermediate degradation, disruption, or instability, and to explain where it diverged from better-scoring reference trajectories.

This thesis takes that missing comparison as its starting point. For a recorded refactoring session, it scores the developer's trace and builds a higher-scoring reference path between the same start and end states, so the two can be compared directly. This leads to

the overall thesis question: *given the start and end state of a refactoring session, can we judge the quality of the path the developer took and point to places where a better path was available?*

Chapter 3

Methodology

This chapter explains how the proposed process-evaluation approach is defined, implemented, and evaluated. It begins by setting out the state/action/trace notation used throughout the chapter (§3.1). It then defines the process-quality score (§3.2.1–§3.2.3) and describes the event-capture, shadow-repository, and metrics infrastructure that supplies the data for that score (§3.2.4–§3.2.6). Next, it introduces the divergence-point detector (§3.3), the Eclipse JDT substrate used to construct alternatives (§3.3.1), the per-kind alternative trajectory synthesisers (§3.3.2–§3.3.5), and the validator used to check that each alternative still reaches the same end state (§3.3.6). The chapter then describes the overall system architecture and offline/replay pipeline (§3.4.1–§3.4.2), followed by the participant dashboard (§3.5). Finally, it defines the evaluation methodology used in Chapter 4: the score-formula sensitivity and ablation design (§3.6), the 45-session dataset and labelling protocol (§3.7), and the divergence-detector evaluation protocol (§3.7.6).

3.1 Formal definitions and notation

The formal definitions in this section build on the state/action/trace framing from §2.1. A refactoring trajectory is a sequence

$$\tau = (s_0, a_0, s_1, a_1, \ldots, a_{T-1}, s_T),$$

where each s_t is a code snapshot and each a_t is a developer action that turns s_t into s_{t+1}. Actions are either IDE-supported refactoring operations or manual edits; in practice a single action can carry both a structured refactoring spec (recovered by RefactoringMiner [7]) and some leftover unstructured edits. We treat this as one action with two parts rather than two separate actions.

The tool evaluates a trajectory by producing an *analysis report*. For a trajectory τ, this report is written as the tuple $(\tau, \mathcal{M}, \mathcal{D}, \mathcal{A})$, where:

- $\mathcal{M} = \{m_0, m_1, \ldots, m_T\}$ is the set of metrics collected for the states in the trajectory. Each m_t contains the cleanliness sub-score $C(s_t)$ and the raw signals used to compute it, including PMD violations, CPD duplications, Chidamber–Kemerer measures, build/test outcomes, the action's refactoring spec, and commit and test event metadata.
- $\mathcal{D}$ is the set of divergence points the detector emits for τ (§3.3).
- $\mathcal{A}$ is the set of alternative reference trajectories synthesised at those divergence points (§3.3.2 onwards). Each $\tau^* \in \mathcal{A}$ is represented in the same way as the observed trajectory, which allows both trajectories to be scored using the same process-quality score.

The process-quality score $J(\tau)$ is computed over the full trajectory using $\mathcal{M}$, and is the focus of §3.2.1–§3.2.2. Table 3.1 summarises the notation used throughout the chapter.

Symbol	Meaning
$\tau = (s_0, a_0, s_1, \ldots, a_{T-1}, s_T)$	Refactoring trajectory
s_t	State (code snapshot) at step t
a_t	Action at step t (IDE refactor or manual edit)
$J(\tau)$	Process-quality score, clamped to $[0, 100]$
$C(s)$	Cleanliness sub-score of state s
$\Delta C(\tau) = C(s_T) - C(s_0)$	Cleanliness gain across trajectory
$\beta = 50$	Baseline anchor in the process score
W_{X}	Process-score weight for term X
$r_b,\ r_{st},\ r_{mi}$	Laplace-smoothed rate terms (broken, skip-tests, manual-when-IDE)
$n_{cg}(\tau)$	Commit-gap event count
$\sigma_l(\tau)$	Step-savings bonus for alternative trajectories
DP	Divergence point
τ^*	Alternative reference trajectory
$\mathcal{K}$	Divergence-kind set = {*Ordering*, *Manual-Refactor*, *Rework*, *Hygiene*}

Table 3.1: Notation used throughout the methodology chapter.

3.2 Computing the process score

This section defines the process score $J(\tau)$ and the cleanliness sub-score used inside it (§3.2.1–§3.2.3). It then describes how the tool captures events, reconstructs shadow repositories, and calculates the per-checkpoint metrics needed to compute the score (§3.2.4–§3.2.6).

3.2.1 The process score

The process score $J(\tau)$ combines seven weighted terms across the trajectory and clamps the result to a $[0, 100]$ range so trajectories of different lengths can be compared on the same scale:

$$J(\tau) = \text{clip}_{[0,100]}\Big(\beta + W_{\text{G}}\,\Delta C(\tau) - W_{\text{B}}\,r_b(\tau) - W_{\text{ST}}\,r_{st}(\tau) - W_{\text{MI}}\,r_{mi}(\tau) - W_{\text{CG}}\,n_{cg}(\tau) - W_{\text{LAG}}\,\ell(\tau) + W_{\text{L}}\,\sigma_l(\tau)\Big) \tag{3.1}$$

where:

- $\beta = 50$ is the baseline. A trajectory with no cleanliness gain and no process penalties scores 50.
- $\Delta C(\tau) = C(s_T) - C(s_0)$ is the cleanliness gain from start to end (§3.2.3).
- $r_b(\tau) = b_{\text{ms}}(\tau)/e_{\text{ms}}(\tau)$ is the fraction of wall-clock time for which the build or test suite was broken.
- $r_{st}(\tau)$ and $r_{mi}(\tau)$ are event-rate penalties. They use Laplace smoothing, $r_\bullet(\tau) = (k+1)/(n+2)$, where k is the number of penalised events and n is the number of opportunities. If no relevant events have occurred yet, the term is set to zero.
- $n_{cg}(\tau)$ counts commit-gap events. One event is recorded for each stretch of at least six green-refactor checkpoints between user commits.
- $\ell(\tau) \in [0, 1]$ measures intermediate-cleanliness lag. It is the per-checkpoint running mean of $\max\big(0,\ C(s_T) - C(s_i)\big)$ over the trajectory, in scalar cleanliness units. This

term is positive while the trajectory remains below its final cleanliness level, so it penalises paths that spend longer in degraded intermediate states.

- $\sigma_l(\tau)$ is a step-savings bonus for synthesised alternatives. It contributes only when τ is an alternative trajectory that reaches the same end state in fewer steps than the observed user trajectory.

Three design choices shape this formula. First, the penalty for a broken build or test suite must be larger than any plausible single-step cleanliness gain. This prevents a trajectory from being rewarded overall for improving structure while temporarily breaking correctness. The weight ordering $W_{\text{B}} : W_{\text{ST}} : W_{\text{CG}} = 28 : 14 : 7$ encodes this priority in §3.2.2. Second, Laplace smoothing prevents the event-rate terms from being dominated by one early event on a very short trajectory. Third, clamping the score to $[0, 100]$ makes trajectories easier to compare, but it also introduces saturation: when a user's trajectory is already close to 100, the improvement available to a higher-scoring alternative is compressed by the ceiling. The sensitivity experiment in §4.1.1 treats this as a known limitation of the score formula.

The score is intended as a consistent and literature-grounded way to compare refactoring trajectories, not as a uniquely correct measure of process quality. There is no held-out ground truth for "true refactoring-trajectory quality" against which the weights could be calibrated directly. The empirical evidence in Chapter 4, including the sensitivity sweep, ablation study, and inter-rater agreement analysis, therefore evaluates robustness, term importance, and classifier accuracy rather than proving that the chosen weights are optimal.

3.2.2 Process-score weights

Table 3.2 lists the seven weights used in Equation (3.1) and summarises the literature behind each one. This section explains why the terms are included. Their empirical effect on the ranking is examined later through the ablation analysis in §4.1.2, which tests all $2^7 = 128$ weight subsets per fixture and reports both solo and leave-one-out recovery against the full formula.

Term	Weight	Cited justification
W_{G} (gain)	50	Cedrim et al. 2017 [30]; Paixão et al. 2017 [17]
W_{B} (broken)	28	Paixão et al. 2017 [17]; Gautam et al. 2025 [9] (FM #2)
W_{ST} (skip-tests)	14	Murphy-Hill et al. 2009 [5]
W_{MI} (manual)	11	Negara et al. 2013 [31]; Vakilian et al. 2012 [32]
W_{L} (length)	11	Harman & Tratt 2007 [14]; Mohan & Greer 2019 [15]
W_{LAG} (lag)	11	Cunningham 1992 [33]; Kruchten et al. 2012 [34]; Cedrim et al. 2017 [30]; Paixão et al. 2017 [17]
W_{CG} (commit-gap)	7	Tao & Kim 2015 [35]; Di Biase et al. 2019 [36]; Herzig & Zeller 2013 [37]; Kudrjavets et al. 2022 [38]

Table 3.2: Weights used in Equation (3.1) and the literature supporting each term. The empirical contribution of each term is examined in the ablation analysis of §4.1.2.

The rest of this section explains each weight in turn. For two terms, W_{MI} and W_{CG}, the literature supports the underlying concern but does not give numerical evidence for how large those penalties should be. Those limitations are noted in the relevant paragraphs.

W_{G} **(gain, 50).** This is the main positive term in the score: each unit of cleanliness gain at the end of the trajectory adds 50 to $J(\tau)$. Cedrim et al. [30] found that refactoring often leaves structural metrics unchanged or slightly worse, so improvement should be rewarded explicitly rather than assumed. Paixão et al. [17] model the balance between disruption and improvement as something developers actively manage, which motivates treating cleanliness gain and process penalties as competing parts of the same score.

W_{B} **(broken, 28).** This is the largest single penalty term. Paixão et al. [17] show that developers place substantial value on avoiding disruptive intermediate states, even when doing so means giving up roughly a quarter of the available metric improvement. The penalty is time-weighted rather than binary: $r_b(\tau) = b_{\text{ms}}/e_{\text{ms}}$, so a brief broken state is penalised less than a long one. The numerator b_{ms} counts only wall-clock time during which the build or tests are failing, rather than total refactoring time.

Time-weighting also makes the term less sensitive to how finely the trajectory is sampled. The state/action/trace framing in §3.1 allows a state to be a commit, an IDE event, or a developer-defined checkpoint. A binary count of broken states would vary substantially across those choices, whereas wall-clock time spent broken is independent of the sampling rate. This choice is also consistent with Gautam et al.'s RefactorBench analysis [9]: their failure-mode #2 shows that rejecting every intermediate broken state can hurt LM-agent performance, because some broken intermediates are necessary during multi-file edits. The weight is therefore set high enough that a plausible single-step cleanliness gain, $W_{\text{G}}\,\Delta C$, cannot outweigh leaving the build or tests broken.

W_{ST} **(skip-tests, 14).** This term penalises 60-second refactoring windows that finish without a test run. The window size follows Murphy-Hill et al. [5], whose study of real refactoring practice shows that developers tend to run tests after batches of changes rather than after every individual edit. The weight is set to half of W_{B}: skipping tests is weaker evidence of process damage than an observed broken state, but it still indicates reduced feedback during the refactoring process.

W_{MI} **(manual-when-IDE, 11).** This term penalises steps where the developer performs a refactoring manually even though the IDE could have applied the same transformation automatically. Mens & Tourwé [4] note that IDE refactoring tools apply static precondition checks that manual edits may bypass, making the automated route safer in many cases. At the same time, manual refactoring is common: Negara et al. [31] found that, across 23 developers and 5,371 refactorings, developers often performed refactorings by hand even when an automated equivalent was available. Vakilian et al. [32] also show that automated refactorings can be misused, so the term is treated as a graded penalty rather than a hard rule.

No published study that we are aware of directly measures the fault-rate gap between manual and IDE refactoring, so this weight is not fitted to such data. Instead, it follows the severity ordering set out above. The ablation analysis in §4.1.2 shows that `manualIde` is one of the three largest single-term contributors by sum-over-sum recovery to session-wide divergence-point magnitude, alongside `length` and `broken` (Table 4.5), and the sensitivity sweep in §4.1.1 indicates that the ranking is robust to perturbations of this weight.

W_{L} **(length, 11).** This is a positive term rather than a penalty. It increases $J(\tau)$ only for alternative trajectories that reach the same end state as the observed user trajectory in fewer steps. Search-based refactoring work treats the number of refactoring operations as an optimisation objective alongside quality [14, 15], and later many-objective work continues this by trading effort against code-quality objectives [23]. The observed user trajectory

is not penalised for length, since a longer path may reflect a larger or more complex task rather than a worse process. The term therefore rewards shorter valid alternatives without treating every long trajectory as poor.

W_{LAG} (lag, 11). This term penalises trajectories that spend time in a worse cleanliness state before reaching their final value. The gain term $W_{\text{G}}\,\Delta C$ only compares the start and end states, so two trajectories receive the same gain score if they finish with the same cleanliness improvement, even if one cleans up early and the other leaves the code degraded until the end. The lag term adds a cost for this delay.

The motivation is similar to technical debt. Cunningham's debt metaphor [33] and Kruchten et al.'s later formalisation [34] both treat degraded code as something that becomes more costly the longer it remains. Cedrim et al. [30] give empirical support for applying this idea to refactoring sequences: they show that intermediate states can degrade structural metrics relative to the start, even when the final result is an improvement. The lag term therefore penalises time spent below the trajectory's final cleanliness target, rather than only measuring the endpoint gap.

The weight is set to 11, the same as W_{L} and W_{MI}, and half of the broken-state penalty W_{B}, following the severity ordering in §3.2.2. Since $\ell \in [0, 1]$, the maximum lag penalty is 11 points, which keeps it smaller than the maximum gain credit of 50. The ablation results give the strongest support for this term in the *Ordering* category: §4.1.2 reports its solo magnitude-recovery, and §4.2.2 shows that the number of *Ordering* divergence-points where the synthesised alternative beats the user's trajectory rises from 4 to 10 of 24 once W_{LAG} is included.

W_{CG} (commit-gap, 7). This term penalises long runs of green checkpoints, where the refactoring checks and tests pass, without a commit in between. The aim is to capture a small process cost: when several valid steps are left bundled together, the resulting change is harder to review, explain, or roll back than if the work had been split into smaller confirmed units.

The evidence comes from work on decomposing composite changes. Tao & Kim [35] report that separated commits let reviewers inspect changes more effectively in the same amount of time. Di Biase et al.'s controlled experiment [36] points in the same direction: decomposed changes produced fewer comments that incorrectly marked correct code as defective (1 rather than 6, $p = 0.03$). Tangled commits can also make later defect attribution less reliable [37].

This is deliberatley a narrow claim. Kudrjavets et al. [38] find no correlation between PR size and time-to-merge across 1.25M PRs in ten languages, so the term is not meant to capture merge speed. It is instead used as a smaller hygiene signal for review comprehensibility and rollback locality. Because no published study that we are aware of directly links commit cadence during refactoring to these costs, the weight is not fitted to data. The ablation results are consistent with this lower-severity role: `commitGap` recovers non-zero session-set magnitude on its own (sum/sum 0.126), and removing it changes the ranking (mean $\tau = 0.926$) while leaving most total magnitude intact (sum/sum 0.892).

3.2.3 The cleanliness sub-score

The cleanliness sub-score $C(s)$ in Equation (3.1) gives each codebase state a normalised quality value in $[0, 100]$. It is built from six signals rather than one, following the argument in §2.3 that individual quality measures are noisy and context-dependent. Combining several signals makes the score less dependent on any single metric. Table 3.3 lists the six sub-signals, the source used to justify each one, and how each value is computed.

Sub-metric	Weight	Cited basis	Computation source
Cognitive complexity	1.0	Campbell 2018 [39]; Muñoz Barón et al. 2020 [40] (partial validation); Lavazza et al. 2023 [41] (contradicting)	Per-method mean
Coupling (CBO)	1.0	Chidamber & Kemerer 1994 [42]	CK library, per-class mean
Duplication	1.0	Heitlager et al. 2007 [43]; Fowler 1999 [2]	CPD-detected cloned lines on touched files
Readability	1.0	Buse & Weimer 2010 [44] (feature taxonomy only - not the trained model)	Five-feature blend, line-weighted
Smells	1.0	Marinescu 2004 [45]; Arcelli Fontana et al. 2016 [20]; Moha et al. 2010 [18]	PMD violation count on touched files
Cohesion (TCC)	1.0	Bieman & Kang 1995 [46]	Per-class mean, dropped if < 2 methods

Table 3.3: The six sub-signals used to compute the cleanliness sub-score $C(s)$. Each is given weight 1.0; the rationale for using equal weights is given in the composition rule below.

Composition rule and sub-weights. Each raw sub-signal is min–max normalised against the main trajectory's range and inverted for "lower is better" measures (cognitive complexity, coupling, duplication, smells). Sub-signals with degenerate ranges or missing data on the trajectory are dropped and the remaining weights are renormalised. Alternative-trajectory checkpoint values are clamped to $[0, 1]$ against the main trajectory's range, so an extreme alternative cannot shift the main trajectory's normalisation. The final cleanliness score is then computed as a weighted sum and scaled to $[0, 100]$.

All six sub-weights are set to 1.0. Weighted sums are widely used in quality models such as Quamoco [47] and QMOOD [48], but those models do not tell us how these particular signals should be balanced across a refactoring trajectory. Existing calibrated composites, such as Coleman et al.'s Maintainability Index [49], are fitted to absolute metric values on single snapshots, not to normalised changes across checkpoints. They therefore do not transfer directly to this setting. In the absence of trajectory-level calibration data, we use equal weights as the least assumptive choice. The sensitivity experiment in §4.1.1 supports this decision: perturbing the cleanliness sub-weights changes only a small fraction of divergence-point rankings, an order of magnitude fewer than the process-score perturbations.

Caveats. Two of the sub-signals have weaker empirical backing than the others:

- *Readability.* We use Buse & Weimer's feature taxonomy [44] (line length, indentation, identifier characteristics) as a feature blend, but not their trained readability model. Calibrating the trained model for trajectory evaluation would require domain-specific data that we have not collected. For this reason, readability is used only as one component of the broader cleanliness score.

- *Cognitive complexity.* The evidence for cognitive complexity is mixed. Muñoz Barón et al. [40] re-analysed 24,400 human understandability ratings and found positive correlations with comprehension time and subjective ratings, although the link with correctness was weak. Lavazza et al. [41], by contrast, found that cognitive complexity predicts understandability about as well as traditional size and cyclomatic-complexity measures, with little extra explanatory value. We therefore treat it as a useful but limited signal within the six-part score.

Coupling to the process score. The process score uses the cleanliness sub-score in two places. First, it uses the endpoint change $\Delta C(\tau) = C(s_T) - C(s_0)$. Second, it uses the final cleanliness value $C(s_T)$ as the reference point for the lag term $\ell(\tau)$. This means that changing a cleanliness sub-weight can affect both the endpoint reward and the lag penalty. This is intentional: the lag term only makes sense if it is measured in the same cleanliness units as the rest of the score. It also has a visible effect in the sensitivity analysis. On the agent sessions, perturbing cleanliness weights by $\times 0.5/\times 1.5$ lowers Kendall's τ by 3–10 % relative to the pre-lag formula (§4.1.1). The injection and user-study sessions do not show measurable rank disruption from the same perturbations.

The process score and cleanliness sub-score are computed from the state of the codebase at each checkpoint in a recorded session. The next three subsections describe how those checkpoint states are produced and measured: event capture in the IDE plugin (§3.2.4), shadow-repo reconstruction that materialises the codebase at each checkpoint (§3.2.5), and per-checkpoint metrics calculation (§3.2.6).

3.2.4 Event capture

The IntelliJ plugin records the events emitted during a user's refactoring session. These include editor events (files opened, closed, saved, modified, renamed, and moved), refactoring events (refactoring start and finish), build events, test-run events with pass/fail counts, git events for newly observed commits, and session start/end events.

File modification activity is the noisiest source. IntelliJ emits an event for every keystroke, so we debounce these events before storing them. The debouncing layer groups keystrokes into an *edit burst* across all files: once there has been no further typing for two seconds, it emits a single burst event containing the file's post-edit contents and the structured set of program elements touched.

Some fine-grained file-level events, such as file-open and file-save events, are not used by the current analysis. We still record them because they are cheap to capture in the plugin and may be useful for later work on navigation patterns, file engagement, and read-versus-edit behaviour.

Before downstream processing, the captured events are re-sorted by timestamp and spurious events are removed. The resulting *normalised event stream* is consumed by every subsequent stage.

3.2.5 Shadow repo reconstruction and worktree pool

Analysing a user's trajectory requires the ability to reconstruct, and cheaply check out, the exact state of the codebase at any point during the session. We do this by replaying the normalised event stream into a fresh per-session git repository, with one commit per state-bearing event. The baseline commit is the initial source snapshot taken at session start; each later event that carries file changes applies its snapshots to the working tree, stages them, and commits. No-op events – those that carry no file changes, or whose staged diff is blank-line-only – map to the previous SHA without producing an empty commit. The output is a per-event map from event id to SHA, which gives every downstream stage a stable (from-SHA, to-SHA) pair to check out.

3.2.6 Metrics calculation

For every unique state in the shadow repo, the pipeline computes a checkpoint metric record. This record contains the build and test outcome, the cleanliness signals (PMD violations, CPD duplications, code readability, and Chidamber–Kemerer structural measures), and the file-level diff against the previous checkpoint.

Build and test outcomes are produced by checking the project state out into a worktree and running the project's own Gradle build and test tasks. For each checkpoint, the pipeline records whether the build succeeded and whether the test suite passed. These outcomes are computed for every checkpoint, regardless of whether the user ran the build or tests during the original session.

This stage dominates the wall-clock cost of the pipeline, so the implementation reuses as much build state as possible. All checkpoints share a single persistent worktree, which is the only stage that bypasses the fresh-per-borrow rule of §3.2.5. The pipeline then walks the unique SHAs in order using detached checkouts. Because the worktree's build directory is preserved across SHAs, the Gradle daemon, Gradle build cache, and incremental compiler can all reuse prior state at the next checkpoint.

The static-analysis signals are tracked across checkpoints rather than only computed independently at each checkpoint. Each PMD violation and CPD clone group is threaded through the unified diffs, allowing the report to identify when a violation was introduced, when it was resolved, and which transition was responsible. This is what lets the dashboard show, for any refactoring step, the smells it introduced or eliminated. Two user-study participants found this inspection feature valuable during interviews (§4.3.1).

3.3 Divergence points

A *divergence point* is a step where the user's trajectory can be compared with an alternative path that scores strictly higher under the process score. The detector reports four kinds of divergence:

$$\mathcal{K} = \{\textit{Ordering},\ \textit{Manual-Refactor},\ \textit{Rework},\ \textit{Hygiene}\}.$$

Each reported point records the divergence *kind*, the *anchor step* where it occurs, and its *magnitude*. It also stores the context needed to explain that kind of divergence: the reorder window for *Ordering*; the replaced refactoring identifier for *Manual-Refactor*; the originating–terminal step pair, file, scope, and reverted-line count for *Rework*; and the hygiene sub-kind and stretch length for *Hygiene*.

The magnitude is uniform across all four kinds:

$$\text{magnitude} = J(\tau^*_{\text{final}}) - J(\tau_{\text{final}}),$$

where τ^*_{final} is the alternative path substituted into the user's trajectory at the divergence point, with the rest of the user's later activity left unchanged. τ_{final} is the score of the user's actual trajectory. Both scores are therefore compared at the same final point, which makes the magnitude comparable across the four divergence kinds: *"if you had taken this one different path at the divergence point and continued with the rest of your trajectory unchanged, by how many process-score points would your final score have moved?"*

Before emitting a divergence point, the detector applies filters specific to each kind. Three thresholds are used. $\textit{min\_rework\_lines} = 2$ removes single-line *Rework* noise. $\textit{min\_commit\_gap} = 6$ requires at least six green-refactor checkpoints between commits before *Commit-Gap* is reported. The 60 s composite-window boundary [5] defines the refactoring batch boundary used by both *Ordering* and *Hygiene*. The value 6 is based on Murphy-Hill's batching heuristic rather than on a measured cutoff from prior work. Since the 60 s window already groups tightly clustered refactorings, requiring six consecutive green checkpoints helps distinguish a real no-commit gap from normal in-batch activity. No published study calibrates this exact threshold, so it is treated as a tunable parameter. The sensitivity analysis in §4.1.1 tests the associated weight W_{CG}, not the threshold itself.

The remainder of the section explains each divergence kind together with the synthesiser that constructs its alternative trajectory. §3.3.1 introduces the Eclipse JDT support used

for *Ordering* and *Manual-Refactor*. §3.3.2–§3.3.5 then cover the four divergence kinds, followed by the bracket-level validator for reordered trajectories in §3.3.6.

3.3.1 Applying refactorings via Eclipse JDT

Synthesising counterfactual trajectories requires the system to apply refactorings, not just describe them. Rather than building a refactoring engine from scratch, the implementation uses Eclipse JDT. This choice was primarily practical: JDT can be embedded as a library and driven programmatically from a JVM, whereas using IntelliJ's refactoring engine would require a custom plugin running inside a full IDE instance, adding UI dependencies, process boundaries, and startup overhead. JDT also provides the refactoring operations needed for this analysis through the same mature implementation used inside Eclipse.

Although JDT was designed for use inside Eclipse, it can also be embedded in another JVM by hosting the Equinox OSGi framework. This is necessary because JDT refactorings depend on Eclipse's workspace model, not just on the Java parser or AST library. The simpler bare-classpath setup is enough for parsing and AST analysis, but not for applying refactorings [50].

Batch sessions. JDT maintains an indexed project model in memory. Building this model takes roughly a second, so repeatedly rebuilding it between refactorings would be expensive. Instead, the pipeline wraps a whole refactoring window in a single *batch session*: the model is initialized once at the start, reused across all operations, and torn down at the end. Each refactoring then sees a fully indexed model without paying the initialization cost repeatedly.

Anchor resolution. Refactorings need to identify the code element they should act on, but raw line numbers are fragile because they change when nearby code is edited. The pipeline therefore uses content-addressed anchors. RefactoringMiner's line and column coordinates are used only once, on the host side, to find the relevant AST node. That node is then represented by more stable information: the fully qualified name of the enclosing type, the host method name and parameter types, and a canonical hash of the node's own AST subtree. This anchor is what the host sends to the JDT bundle. For range-based operations, the original line and column are not sent at all; for point-based operations, they are kept only as a tiebreaker in the rare case where two declarations inside the same method have the same subtree hash. The JDT bundle first finds the host method by name and parameter types inside the named type, then searches within it for a node with the matching subtree hash. Because both sides compute the hash in the same way, the lookup is robust to formatting changes and to line shifts elsewhere in the file. For example, a method that moves from line 42 to line 47 still resolves correctly, as long as the method's own subtree has not been rewritten.

3.3.2 Ordering

Ordering detection is applied to bracket-validated windows of consecutive refactoring steps (§3.3.6). For each window, the reorder synthesiser computes the valid permutations of those steps and measures how each ordering changes the process score relative to the developer's original order. The system emits at most one divergence point for the window, placed at its final step. Its magnitude is the best score improvement found among the admitted permutations, and the admitted permutations are attached as alternatives.

The reorder synthesiser produces these alternatives by reshuffling the user's refactoring steps while still requiring the final code state to match the user's. The following paragraphs

describe how candidate reorderings are generated, filtered, and checked before they are scored.

Window splitting on validator verdicts. Only bracket-validated windows (§3.3.6) are eligible for reordering. A bracket is rejected if the synthesiser cannot reproduce it. This can happen in three ways: *untyped* means that RefactoringMiner could not map the user's edit to a typed JDT refactoring spec, which is common for unstructured manual edits; *refactor_failed* means that JDT tried to apply the spec but failed; and *ast_diverged* means that JDT applied the spec, but the resulting AST did not match the user's. If any of these verdicts occurs, the containing bracket is split into individual steps that are not reordered. The wrap-and-patch layer (§3.3.3) handles the most common JDT-IntelliJ differences before this validation step, increasing the number of brackets that pass.

Dependency-DAG construction. Each refactoring step carries a structured *spec* describing what entities it reads, writes, creates, or deletes. The synthesiser builds a dependency graph where an edge from step u to step v means v depends on u's effects. This graph defines the constraints on any valid reordering.

SSA versioning across renames. Renames make dependency tracking harder because the same program entity can appear under different names at different points in the trajectory. For example, if a method is renamed from `foo` to `bar`, later steps refer to `bar`, even though it is the same method. The synthesiser handles this by assigning a new version identifier after each rename and carrying that identifier through the dependency graph. Later steps that target the renamed entity are then linked to the correct version. This is needed for cases such as Extract Method followed by Rename Method on the extracted method.

Topological enumeration. The synthesiser enumerates valid topological orderings of the dependency graph using recursive backtracking. Each ordering is one candidate permutation of the window's refactoring steps. Because the number of possible orderings can grow very quickly, the search has a hard size limit: windows with more than 7 refactoring steps are skipped, and enumeration within an accepted window stops after at most $7! = 5040$ permutations. In the recorded sessions used in Chapter 4, reorder windows rarely reached this limit. However, sessions with longer uninterrupted refactoring sequences could have their largest windows skipped by the *Ordering* detector; this limitation is discussed further in §5.2.3.

Depth-first search over a prefix trie. The enumeration is implemented as a depth-first search over a prefix trie of candidate permutations. This lets the synthesiser reuse work across orderings that begin with the same sequence of refactorings. When the search moves forward, it applies one refactoring to the worktree; when it backtracks, it rolls the worktree back to the previous state. As a result, two orderings that share their first j steps also share the work needed to materialise those j steps, instead of applying the same prefix repeatedly from scratch.

Terminal AST-equivalence audit. For each full permutation, the synthesiser checks that the final code state matches the user's final code state. This is the correctness check for reordering: a candidate ordering is only kept if it reaches the same end point as the original trajectory. At each leaf of the trie, meaning the terminal state of one complete ordering, the synthesiser hashes the ASTs of the touched files into a canonical form. These hashes are compared with the user's terminal AST hashes, which are precomputed from

the user's own trajectory with `git show` at the start of the window, so a second worktree is not needed. The canonical form ignores differences that should not affect behaviour, such as whitespace, comment position, formatting, and method-declaration order within a class. The method-order case matters because applying an Extract Method earlier or later in the bracket can place the new method at a different lexical position even when the resulting class is otherwise the same. If the terminal hashes match the user's, the ordering is kept and scored under Equation (3.1); otherwise it is discarded.

Three implementation choices keep this search practical. First, the whole window runs against a single borrowed worktree. Forward steps, backtracking, and the final AST check therefore reuse the same checkout instead of creating a fresh worktree for every candidate ordering. Second, the window runs inside a single JDT batch session (§3.3.1), so each refactoring can reuse the already-initialized project model. Third, when the search backtracks, the worktree is rolled back with a detached git checkout and JDT is asked to refresh its view of the changed files, just as it would after external file changes in an IDE.

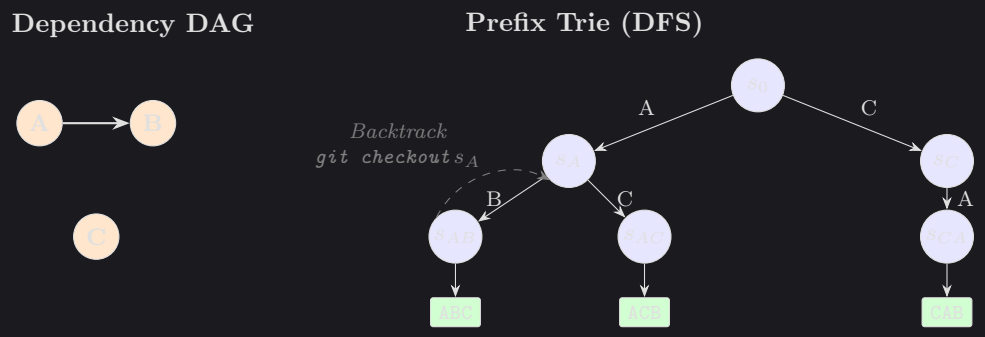

Figure 3.1: Reorder synthesis for a three-step window with one dependency edge ($A \rightarrow B$). The dependency graph on the left permits three valid orderings. The prefix trie on the right shows how shared prefixes are reused: the prefix A is applied once, then reused by both orderings that begin with A.

3.3.3 Manual-Refactor

A *Manual-Refactor* divergence point is emitted at any user step where (i) the action was performed by hand, and (ii) the resulting diff matches the structural pattern of an IDE-supported refactoring. IDE-driven refactorings arrive pre-tagged in the captured event stream, but a manual refactoring is just a series of edit bursts whose net effect happens to constitute a structured refactoring. The pipeline recovers them by running RefactoringMiner [7] – a standard tool that detects refactorings between two code states by AST-level structural matching – over a sliding window of commit pairs in the shadow repo. The window matters because a manual refactoring may unfold across several intermediate edit bursts, each of which on its own looks like an unrelated change; only by widening the window to cover the full span does the refactoring become visible to the miner. Each detected refactoring is then cross-checked against the captured event stream: a detection whose window contains a matching IDE-emitted refactoring event is recorded as IDE-driven, and one whose window contains no such event is flagged as manual. Adjacent candidate steps that share the same `(fromSha, toSha)` bracket are grouped into a single composite that the synthesiser treats as one refactoring.

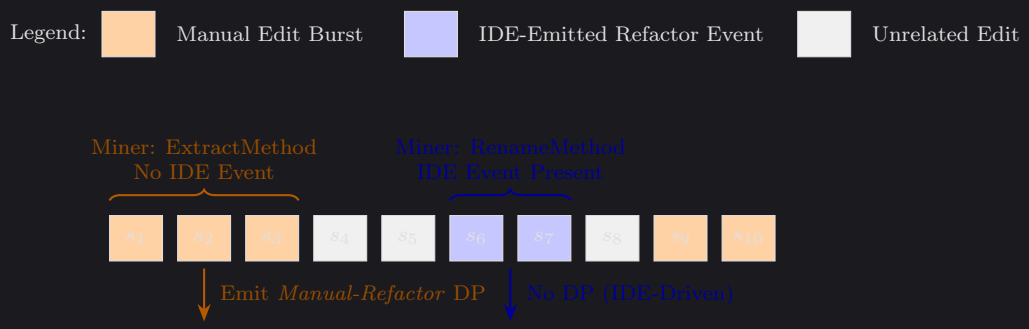

Figure 3.2: Manual-Refactor detection over a sliding edit window. RefactoringMiner detects structured refactorings between bracket endpoints, and each detection is checked against the IDE event stream. A detection with no matching IDE event is treated as manual and emits a divergence point; a detection with a matching IDE event is recorded as IDE-driven.

The manual-refactor synthesiser applies each detected refactoring spec through the corresponding IDE-style operation in JDT. It then runs a residual three-way merge to add back the user's other edits, such as bug fixes or small unrelated changes that were interleaved with the refactor. The resulting tree is committed to the shadow repository. This merge step is needed so that the alternative trajectory keeps the user's non-refactoring edits and still ends at the same code state as the original trajectory. The comparison then measures the difference between doing the refactoring by hand and doing it through the IDE, rather than being distorted by missing edits in the final state. Without this merge, the alternative trajectory would diverge from the user's final code and the process-score comparison would not be meaningful.

The wrap-and-patch layer. One practical issue is that sessions are recorded in IntelliJ, while the pipeline replays refactorings through JDT. The two engines can produce slightly different ASTs even when they are carrying out the same refactoring. To reduce these false divergences, the pipeline includes a wrap-and-patch layer between the JDT output and the validator. During testing, two recurring differences were observed. First, JDT adds the `static` modifier to an extracted method only when the method body requires it, whereas IntelliJ adds it whenever the body permits it. Second, JDT sometimes under-detects outer-scope-variable liveness in conditional branches, producing a void-returning method where IntelliJ produces a value-returning one. A host-side post-pass detects these patterns and rewrites the JDT output to match IntelliJ's structure. If a remaining difference cannot be patched safely, the bracket is marked as diverged and is not reordered. The wrap-and-patch layer therefore increases the number of usable brackets, but it does not cover every possible JDT-IntelliJ difference; full coverage is left as future work and discussed further in §3.3.6.

3.3.4 Rework

Rework detection looks for edits that are later undone. In practice, this means finding content that is added and then removed, or removed and then added back, so that the two changes cancel out apart from whitespace. The detector parses each step's unified diff into hunks, maps each hunk to its enclosing method or class, and hashes the affected lines after normalising whitespace. It then greedily pairs added and removed hunks that share the same `(file, scope, content-hash)`. Each matched `(originating step, terminal step)` pair becomes a *Rework* candidate, while pairs below the *min_rework_lines* threshold are discarded as line-level noise.

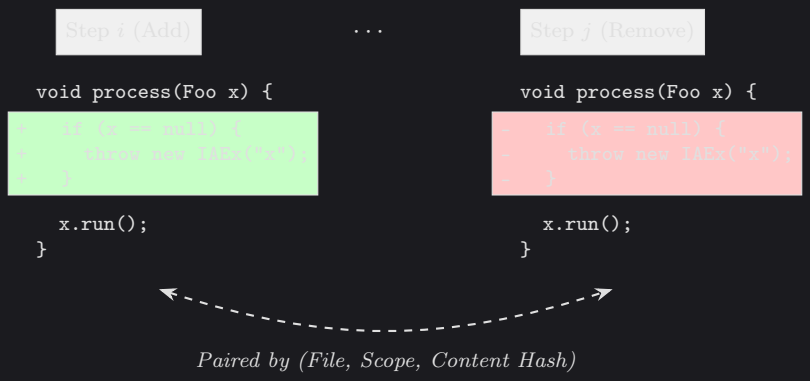

Figure 3.3: Rework detection for an added chunk that is later removed. The detector pairs the two hunks using their `(file, scope, content-hash)` *tuple. The synthesiser then builds a shorter alternative trajectory by removing both halves of the rework and preserving the other intermediate edits.*

The rework synthesiser builds an alternative trajectory that skips the matched originating–terminal hunk pair. For each pair, it replays the [originating, terminal] window as a sequence of small patches, leaving out the reworked chunk while preserving the other changes in the window. The replay is treated atomically: if any patch fails to apply, the whole alternative is discarded, so the system never scores a partially reconstructed trajectory.

A key issue is that the intermediate edits have shifted line numbers around the chunk. The terminal-step patch therefore has to translate its line coordinates from the user's tree, where the chunk reappears, to the synthesised tree, where the chunk was never written and the file may be several lines shorter. To do this, the synthesiser keeps a per-file drift tracker. It records the line offset between the two trees as a set of zones, and each intermediate user hunk shifts the zones after it by that hunk's net line change. This keeps each zone tied to the same logical content as the user's tree grows or shrinks around it.

After the reworked chunk is removed, some intermediate checkpoints no longer contain any substantive change: their only edit was the content that has now been omitted. Others contain only whitespace changes, such as blank-line edits around the removed chunk. Empty checkpoints are dropped from the alternative trajectory instead of being committed as no-op steps, while checkpoints whose remaining change is purely whitespace are folded into the preceding checkpoint. Both cases shorten the alternative's step count, which improves its process score through the step-savings term defined in §3.2.1.

3.3.5 Hygiene

Hygiene detection covers two cases where the refactoring process remains technically valid but lacks a useful safety checkpoint. *Tests-Skipped* is reported once for each 60-second composite window [5] that contains at least one refactoring step but no successful test run. *Commit-Gap* is reported once for each stretch of at least six green-refactor checkpoints, meaning checkpoints where the tests passed and the refactoring applied cleanly, without an intervening user commit. Each finding emits one divergence point.

The hygiene synthesiser builds alternatives in which the user ran tests or made a commit at the relevant checkpoint, without changing the code. For *Tests-Skipped*, the alternative adds a synthetic `TEST_RUN_FINISHED` event at the anchor checkpoint. It does not change the recorded test outcome: `tests.success` and `tests.wasSkipped` remain as they were in the user's trace. The counterfactual is therefore "the user should have run the tests here", not "the tests would have passed here". This affects W_{ST}, which depends on whether tests were run, but leaves W_{B} unchanged because that term depends on the broken/passing outcome. For *Commit-Gap*, the alternative adds a commit at the anchor checkpoint,

breaking what would otherwise have been a long stretch of uncommitted green-refactor checkpoints.

In both cases, the alternative changes only the process metadata at the anchor checkpoint, not the code. This is because hygiene concerns the process around the refactoring: when tests were run and when commits were made. Synthesis therefore amounts to flipping the relevant process flag at the anchor checkpoint and feeding the trajectory back through Phase B (§3.4). The difference between the original and adjusted final scores gives the process-score cost of the skipped hygiene event.

3.3.6 Validator and behaviour preservation

The bracket-level validator checks whether the user's specs can be replayed in order to reach the same final state as the user's trajectory. For each `(fromSha, toSha)` bracket, it replays the specs and then applies two checks. First, the set of changed `.java` files must match `git diff fromSha..toSha`. Second, the terminal AST hash for each changed file must match the user's version. Brackets that pass both checks are marked as *valid* and can be used as reorder windows. Brackets that fail are split into individual steps and are not reordered. The replay uses the same borrowed worktree and JDT batch-session machinery as the reorder engine (§3.3.2).

The number of brackets that pass validation is limited by differences between JDT and IntelliJ, as discussed in §3.3.3. If the wrap-and-patch layer covers the difference, the bracket can still be marked as valid. If it does not, the bracket fails validation even when the original refactoring was correct. Covering all structural differences between JDT and IntelliJ is left as future work, and this limitation is the main reason *Ordering* recall is not higher in §4.2.2.

3.4 System architecture and pipeline phases

This section describes the four JVM modules that make up the tool and explains how they fit into the analysis pipeline. §3.4.1 gives the module-level overview and dataflow, while §3.4.2 explains the Phase A / Phase B split used by the implementation.

3.4.1 Architecture at a glance

The tool is split into four JVM modules and a React/TypeScript dashboard:

- `ide-plugin/` – an IntelliJ plugin that records the developer's session as a stream of typed events and uploads it to the analysis server. The user starts and stops recording with a button in the IDE.
- `analysis/` – the Kotlin analysis pipeline. It ingests recorded sessions, reconstructs a shadow git repository, mines and synthesises trajectories, computes per-checkpoint metrics, and emits an `AnalysisReport`.
- `refactoring-bundle/` – a headless Eclipse JDT distribution wrapped in an embedded Equinox OSGi framework. The analysis module calls into it across the OSGi classloader boundary (details in §3.3.3).
- `dashboard/` – a React/TypeScript single-page app that renders the `AnalysisReport` (details in §3.5).

Figure 3.4 summarises the dataflow between these components.

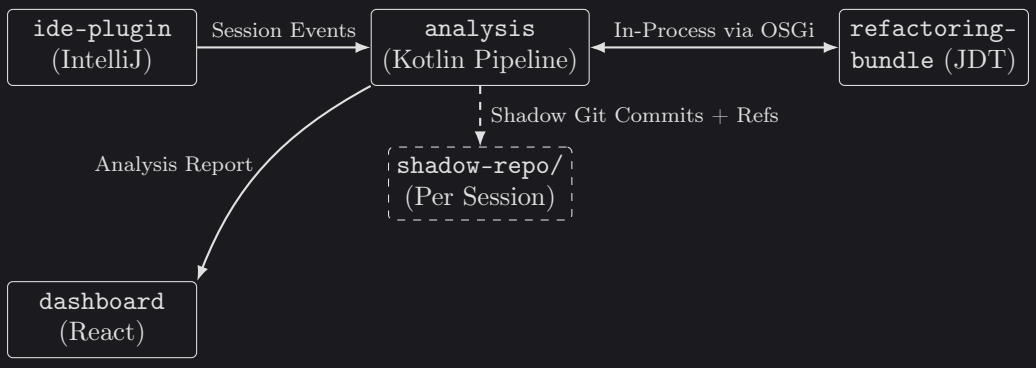

Figure 3.4: High-level dataflow through the tool. The IDE plugin records the session, the analysis pipeline reconstructs and scores the trace, and the embedded JDT bundle applies synthesised refactorings. The dashboard displays the resulting report, while the shadow repository stores user checkpoints and synthesised alternatives as per-session git commits.

3.4.2 Phase A / Phase B split

The detector and synthesisers run as part of an offline analysis pipeline split into two phases. The first phase performs the expensive work: it ingests the recorded events, reconstructs the shadow repository, mines refactorings, runs the synthesisers and validator, computes per-checkpoint metrics, and tracks PMD / CPD findings across checkpoints. This phase usually takes minutes per session, mainly because it runs builds, tests, and JDT refactorings.

Phase A. The output of the first phase is a cached analysis bundle containing the reconstructed trace, mined refactoring steps, per-checkpoint metrics, synthesised alternatives, and supporting diff/quality data. It is saved so that the expensive reconstruction work does not need to be repeated.

Phase B. The second phase takes this cached bundle and a chosen set of score weights. It computes the process score, attaches alternative-vs-user deltas, and produces the final report. Unlike the first phase, it does not reconstruct repositories, apply refactorings, or rerun builds and tests; it only re-scores the cached analysis data, so it finishes in milliseconds.

This split is important for the sensitivity experiments in Chapter 4. Changing the score weights only affects the second phase, so thousands of scoring configurations can be evaluated against the same cached first-phase output without rebuilding trajectories or rerunning tests.

3.5 Dashboard

The detector and synthesisers produce an analysis report, and the dashboard presents that report to participants. The dashboard is a React web app hosted inside the IDE through JCEF, IntelliJ's embedded Chromium browser, in a dialog rather than a separate browser page, and the plugin passes the completed report to the page once analysis has finished. Figure 3.5 shows the dashboard in use on one of the user-study sessions.

The main view is a trajectory chart showing the user's process score over time, with synthesised alternatives overlaid for comparison. Participants can toggle the individual code-cleanliness signals on the chart, such as readability, complexity, duplication, code smells, coupling, and cohesion, to see how different aspects of the code changed across the session. Selecting a step opens a detail panel for the exact edit made at that point, including the affected code, the metrics that changed, and any divergence point associated with the

step. Divergence points are clickable: they explain what went wrong, what alternative action the developer *could* have taken, and how much that alternative would have changed the process score. The dashboard also surfaces the locations of code smells and duplicated code, and shows build and test status over time so participants can see where the session became broken and where it recovered. It also highlights the highest-impact and actionable takeaways from the session as a whole.

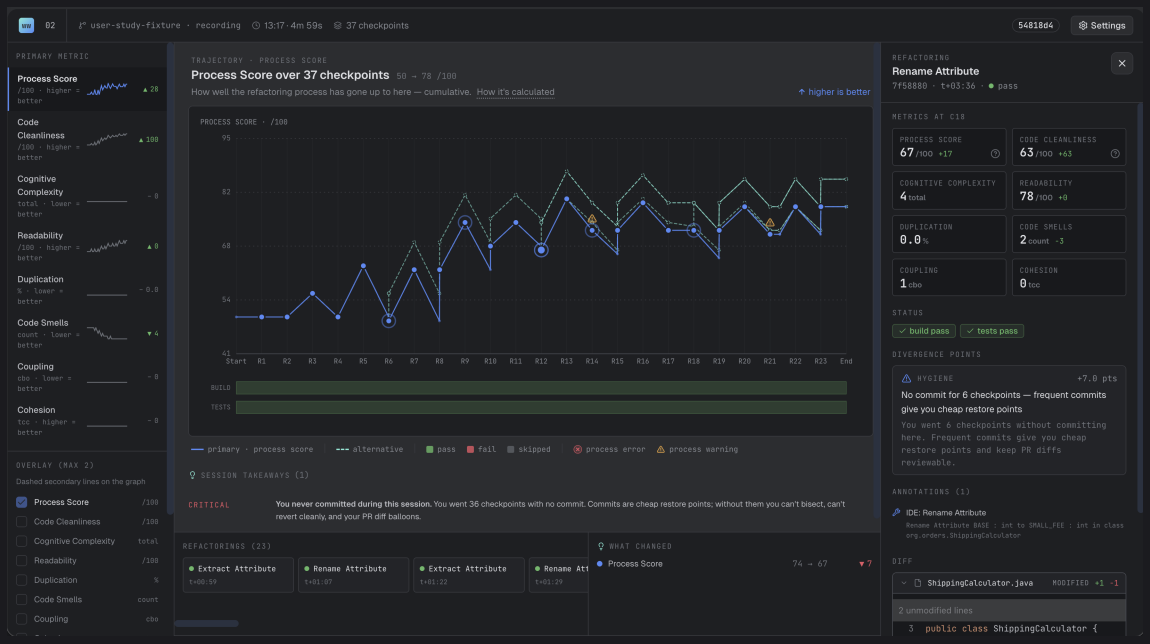

Figure 3.5: The dashboard hosted inside an IntelliJ JCEF window. The centre chart plots the user's process score across the session and can overlay alternative trajectories and individual cleanliness signals. The side panels show the selected step, metric changes, build/test status, smell and duplication locations, and any divergence point associated with that step, including the suggested alternative and its score impact. The lower panel summarises the highest-impact takeaways for the session.

3.6 Score-formula evaluation design

The results chapter evaluates the score formula in two ways. The first asks whether the ranking of divergence points is stable when the weights are perturbed. The second asks whether the individual terms in the formula actually contribute to the magnitudes the detector reports. This section defines the shared experimental procedure for those tests; Chapter 4 reports the observed numbers.

3.6.1 Sensitivity perturbations

The sensitivity experiment compares the ranking of divergence points computed by the production weights with rankings produced under perturbed weights. Two perturbation schemes are used. In the single-knob sweep, one weight is varied while all other weights remain at their production values. In the multi-knob sweep, all weights are varied at once by drawing independent log-normal scale factors. For weight W_i, the sampled value is

$$W_i' = W_i \cdot \exp(\sigma Z_i),$$

where $Z_i \sim \mathcal{N}(0, 1)$ and $\sigma = \ln 2$. This makes the perturbation symmetric in log-space: multiplying and dividing by the same factor are treated as equally large changes, and most samples remain within a few multiples of the production value.

Ranking stability is measured with Kendall's τ-b [51, 52] and top-N hit rate. Kendall's τ-b is used because the rankings are short and may contain ties. A value of $\tau = 1.0$ means the perturbed ranking is unchanged, while $\tau < 0$ means the order has mostly inverted. Top-N hit rate asks how many of the production top-N divergence points remain in the perturbed top-N. Top-1 is the main comparison; larger N values are only informative for sessions with enough surfaced points. Both metrics are mechanically 1.0 on sessions with ≤ 1 detected divergence point — a single-item ranking cannot be perturbed and a zero-item ranking is undefined — so the rank-based statistics in Chapter 4 are computed on the *rankable subset* of each session set: sessions whose analysis report contains ≥ 2 detected divergence points (any magnitude sign). Magnitude-based statistics (mean recovery, sum-over-sum) continue to use the full session sets, since magnitudes are well-defined on every session. Ties are broken by ascending step index so that production and perturbed rankings are compared deterministically.

3.6.2 Ablation design

The ablation experiment tests whether the score formula depends on each of the seven non-sub-cleanliness terms: gain, broken-build time, skipped tests, manual-when-IDE, length, lag, and commit gap. It enumerates subsets of these process weights, keeps the selected terms at their production values, and sets the others to zero. Since there are seven non-sub-cleanliness terms, the full power set contains $2^7 = 128$ variants, which is small enough to enumerate exactly. This follows the same logic as composite-indicator sensitivity analysis [53] and exact Shapley-style feature attribution when the number of features is small [54, 55].

Two recovery measures are used to describe how much divergence-point magnitude remains under an ablated formula. Mean recovery computes the recovery ratio per fixture and averages those ratios. Sum-over-sum recovery instead sums |magnitude| across all divergence points in all fixtures and divides by the same sum under the production formula. Sum-over-sum is treated as the main measure because it avoids giving empty or near-empty fixtures the same weight as fixtures with many surfaced divergence points. Both measures are reported on the full injection set (45 sessions) because magnitude is well-defined on every session regardless of per-session DP count; the cross-set rank comparison in Chapter 4 uses the rankable subset of each set, as defined above.

3.7 Evaluation dataset and labelling protocol

The experiments in Chapter 4 use a set of 45 refactoring sessions that we recorded by hand on a Java library-style fixture in IntelliJ. This section explains how the sessions were assembled, how they were labelled, and how the labels were checked by raters.

The IDE-event capture literature surveyed in §2.6 shows that detailed development traces can be collected. However, we did not find an existing dataset that combines per-event code content with IDE-side refactoring events for Java sessions recorded in IntelliJ. This matters for the IDE-versus-manual signal. RefactoringMiner can recover a structural refactoring from a pair of code states, but it cannot tell whether the developer used the IDE refactoring command or typed the change by hand. Since that distinction is part of the manual-when-IDE penalty in §3.2.1, it has to be captured at recording time.

The evaluation also needs controlled examples of the behaviours the detector is meant to find. The precision and recall results in §4.2.2 are computed against sessions where a specific process issue was deliberately introduced. A free-form session dataset would not provide that controlled injection signal directly. For these reasons, we recorded our own dataset using the IntelliJ plugin described in §3.2.4, so that IDE events were captured as

the sessions happened.

3.7.1 The 45-session injection set

Each session follows a scripted playbook scenario designed to trigger one specific kind of process issue. Examples include choosing a less favourable order for a set of refactoring steps, undoing and redoing the same chunk of code, skipping the expected test cadence, or making a manual edit where an IDE refactoring would have applied. The playbook covers all four divergence kinds: 5 sessions target ordering, 9 target rework, 13 target hygiene, and the remaining sessions are split between IDE-replay injections and clean controls where no issue is deliberately introduced.

3.7.2 Two label streams

Each session has two kinds of label, because the evaluation asks two slightly different questions. The first label records the divergence kind that was deliberately injected into the session, i.e. the behaviour the playbook was designed to trigger. This is used for the *injection prominence* analysis in §4.2.2. The second label is a separate hand-written set of all divergence kinds that should be detected in the session. This label can contain more than one kind, because a session designed to trigger one issue may also legitimately contain another. The per-kind precision and recall results in §4.2.2 use this second label set, and its inter-rater agreement is reported in §4.2.1.

The multi-label schema uses the four divergence kinds $\mathcal{K} = \{\textit{IDE-Replay}, \textit{Rework}, \textit{Hygiene}, \textit{Ordering}\}$ and each session is allowed to have more than one expected kind. For example, a session may contain both an ordering divergence and a hygiene flag.

3.7.3 Rater protocol

To check the schema independently, two annotators, denoted P1 and P2 in the results chapter, labelled all 45 sessions using the same schema. They completed this labelling before either of them took part in a user-study session. Each rater received the schema definition and the raw artefacts for every recording: the event log `events.jsonl` and the session metadata `session.json`. They did not receive the playbook prompts, our `manifest-v2.csv` labels, the Phase A `analysis-report.json`, or access to the dashboard. This kept the hand labels separate from both the injected labels and the detector output.

The event log stores the full file contents after each event rather than a compact diff. To make the sessions easier to inspect, raters were also given a small read-only helper script, `tool/scripts/session-event-diffs.py`, which prints a unified diff for each *edit_burst* and *refactoring_finished* event. The script is only helps to present the information from the events to the users in a more readable format and does not expose any extra information: it derives its output from the same `events.jsonl` file already given to the raters.

3.7.4 Cohen's κ computation

Agreement is reported with Cohen's κ [56], computed separately for each divergence kind in $\mathcal{K}$. For a given kind k, each session is treated as a binary judgement: either k is present in `expected_kinds`, or it is not. This gives a 2×2 table for each rater pair, from which κ is computed. The results are interpreted using the Landis and Koch thresholds [57]: $\kappa < 0.20$ is slight agreement, 0.21–0.40 fair, 0.41–0.60 moderate, 0.61–0.80 substantial, and 0.81–1.00 almost perfect. Because the dataset contains only 45 sessions, the raw agreement counts from each table are reported alongside κ; at this size, the yes/yes, no/no, and disagreement counts are still useful to inspect directly.

3.7.5 Treatment of disagreements

Disagreements are reported as part of the study rather than resolved by changing our labels or the raters' labels. When both raters agree with each other but disagree with our label, as with rework in session 043, the session is named explicitly in the results chapter and treated as a case where our label may be the outlier.

The rater-study results, including the per-kind κ values, raw agreement counts, and disagreement patterns, are reported in Table 4.8 (§4.2.1).

3.7.6 Divergence-detector evaluation protocol

The divergence-detector experiment runs the full analysis pipeline on each of the 45 recorded sessions using the production score weights. For each session, the detector output is compared with the session's `expected_kinds` label. A true positive means that the detector surfaced at least one divergence point of the expected kind somewhere in the session, a false negative means that an expected kind was not surfaced, and a false positive means that the detector surfaced a kind not present in `expected_kinds`. Matching is done at the session level rather than against an exact recorded step, because step boundaries depend on the plugin's edit-burst flushing and are not stable enough to treat as ground truth.

Precision and recall are reported separately for each kind. They are not collapsed into F1 or accuracy because the two error types have different practical meanings: a false positive shows the user an alternative that should not be there, while a false negative means the dashboard missed a useful opportunity. Accuracy is also uninformative because most kind-by-session cases are true negatives.

The experiment also checks whether the detector's output would be useful to a user. The first check is whether a surfaced alternative genuinely beats the user's trajectory, using the strict condition magnitude > 0 under the trajectory-final score definition in §3.3. The second is injection prominence: for the divergence kind deliberately introduced by the playbook, the experiment records whether the corresponding positive-magnitude divergence point is the highest-ranked point in that session.

Chapter 4

Experimentation and Evaluation

This chapter evaluates the score formula, the divergence-point detector, and the use of their feedback in complete refactoring sessions. The formula is tested through sensitivity and ablation experiments, the detector against labelled injection sessions, and the end-to-end workflow through a 30-session human study plus an agent extension on the same codebase.

§4.1 reports the score-formula experiments. §4.2 reports detector precision and recall against externally labelled divergence kinds. §4.3 reports the human and agent studies, where the outcome is whether feedback is reflected in later refactoring behaviour.

The labelled detector experiment uses the 45-session injection set and two-label scheme defined in §3.7. Because the per-kind counts are small, tables report raw counts alongside percentages. Every magnitude value in this chapter is the trajectory-final J delta defined in §3.2.1, so magnitudes are comparable across the four divergence kinds.

4.1 Testing the score formula

The score weights were chosen from a literature-supported severity ordering (§3.2.2), not fitted to an external outcome dataset. This section therefore tests whether the rankings are stable under plausible weight changes and whether the seven process-side terms contribute measurable signal, following the evaluation design in §3.6.

4.1.1 Sensitivity sweep

The first experiment measures how sensitive the divergence-point ranking is to the weights in the process-score formula. Large ranking changes under small weight perturbations would indicate that the order of divergence points depends too strongly on arbitrary parameter choices. The experiment uses the single-knob and multi-knob perturbation methods defined in §3.6.1. The main result is that the top-ranked divergence point is usually stable across all three session types, although some terms do affect the ranking in identifiable cases.

Experiment setup. For the single-knob sweep, the chosen weight is scaled by one of nine factors in $\{0.1, 0.25, 0.5, 0.75, 1.25, 1.5, 2.0, 4.0, 10.0\}$. With thirteen weights (7 process-side and 6 cleanliness-side), this gives $13 \times 9 \times |\mathcal{C}|$ perturbation runs per session set $\mathcal{C}$. The multi-knob sweep draws 200 independent samples per session using the log-normal sampling scheme defined in §3.6.1.

The experiment starts from three session sets: the 45 hand-labelled injection sessions of §4.2.2, the 30 user-study sessions of §4.3.1, and the 48 agent extension sessions of §4.3.2. For the sensitivity analysis, only sessions with at least two detected divergence points are rankable, because sessions with one divergence point have no meaningful ordering to

perturb and sessions with none have no ranking at all. This leaves 15 injection sessions, 25 user-study sessions, and 20 agent sessions in scope for the rank-based statistics reported below.

The three session sets still differ in their rankable-session profiles. The injection sessions have at most 5 detected divergence points per session and concentrate on single-kind exercises. The user-study sessions have up to 15 detected divergence points per session across more natural sessions. The agent sessions have at most 4 detected divergence points per session across the eight agent configurations, partly because the detector is designed around human IDE edit patterns rather than agent file-edit patterns (§4.3.2). The later magnitude-recovery statistics in §4.1.2 use the full session sets, since magnitudes are well-defined even for sessions with zero or one divergence point.

As defined in §3.6.1, ranking stability is reported using Kendall's τ-b and top-N hit rate. Top-1 is defined for every rankable session, so it can be compared across all three session types. Top-3 and top-5 are reported on the user-study sessions, where the per-session rankings are large enough to make them informative. Several divergence-point kinds produce ties, so ties are broken by ascending step index in both the production and perturbed rankings.

Single-knob results. Table 4.1 reports top-1 stability across the rankable subsets of the three session types. The user-study set is the strongest test of ranking stability because it carries the largest rankable subset (25 of 30 sessions), the longest per-session rankings, and a low clamp-frozen rate. The injection rankable subset is the smallest (15 sessions); its top-1 stability is high but rests on a smaller set of rankable sessions. The agent rankable subset (20 sessions) sits between the two, although 20.8% of its $(\text{DP}, \text{perturbation})$ cases are clamp-frozen — the user and alt terminals both sit at the same $[0, 100]$ clamp value in both baseline and perturbed scorings, so the magnitude is identical by construction. On the user-study rankable subset, the top-1 recommendation is preserved in 2,823 of 2,925 perturbation cases (96.5%).

session set (rankable)	cases	$\tau = 1.0$	top-1 $= 1$	$\tau < 0$	clamp-frozen rate
Injection (15)	1,755	30.7%	99.0%	1.3%	8.6%
User study (25)	2,925	41.9%	**96.5%**	1.0%	**1.8%**
Agent (20)	2,340	72.4%	96.3%	3.0%	20.8%

Table 4.1: Single-knob sensitivity across the rankable subsets of the three session types (≥ 2 detected divergence points per session). The clamp-frozen rate is the share of (DP × perturbation) cases where the user trajectory-final and the DP's alt trajectory-final both sit at the same $[0, 100]$ clamp value in baseline and in perturbed; for those cases the magnitude $J(alt_T) - J(user_T)$ is identical by construction, contributing trivially to $\tau = 1$. Only 1.8% of user-study cases are clamp-frozen, so the 96.5% top-1 figure is overwhelmingly genuine perturbation response rather than clamp-driven invariance.

Sources of ranking instability. Table 4.2 reports how often each individual weight changes the top-ranked divergence point on the rankable user-study subset. The main pattern is that almost all top-1 shifts come from the process-side weights. Six of the seven process-side weights change the top-ranked item in 5.3%–10.7% of perturbation cases, with W_{G} having the largest single-weight effect (24 of 225 cases, 10.7%). The exception is W_{LAG}, which changes the top-ranked item only once (0.4%). This matches the leave-one-out result in §4.1.2: the lag term contributes magnitude, but does not noticeably reshape the user-study ranking. The cleanliness sub-weights have even less effect, changing the top-ranked item in at most 0.9% of cases, because they usually move the user and alternative trajectories in similar ways.

knob	top-1 changes / 225 cases	share
`gain`	24	10.7%
`manualIde`	17	7.6%
`skipTests`	15	6.7%
`commitGap`	15	6.7%
`length`	14	6.2%
`broken`	12	5.3%
`lag`	1	0.4%
cleanliness sub-weights: `coupling` 2 (0.9%), `smells` 1, `cohesion` 1, *all others* 0		

Table 4.2: Per-knob top-1 disruption counts on the user-study rankable subset (25 sessions × 9 perturbation factors = 225 cases per weight). Most top-1 shifts come from the process-side weights, especially W_G; the lag weight changes the top-ranked item only once, and the cleanliness sub-weights have little effect.

Multi-knob Monte Carlo. The single-knob sweep shows how the ranking responds when one weight moves in isolation. The multi-knob Monte Carlo perturbs every weight together. Table 4.3 reports the resulting τ and top-N hit rates across $200 \times |\mathcal{C}|$ joint-perturbation samples for each session type.

The joint perturbation test gives a weaker stability result than the single-knob sweep. On the user-study rankable subset, the top-1 recommendation changes on 15.4% of samples, compared with 3.5% under single-knob perturbation, and mean τ falls to 0.586. The injection subset still has high top-1 stability (95.7%), but its mean τ drops to 0.385. This lower τ is partly a consequence of the shorter injection rankings: a joint perturbation can reorder many of the lower-ranked divergence points without changing the top recommendation. The agent subset sits between the two, with mean $\tau = 0.645$ and top-1 stability of 82.5%. Overall, the formula appears stable under small joint perturbations, but the ranking becomes less reliable as several weights move together.

session set (rankable)	samples	mean τ	top-1 = 1	top-3 = 1	top-5 = 1
Injection (15)	3,000	0.385	95.7%	32.9%	33.3%
User study (25)	5,000	**0.586**	**84.6%**	**61.7%**	59.5%
Agent (20)	4,000	0.645	82.5%	78.3%	75.0%

Table 4.3: Multi-knob Monte Carlo robustness across the rankable subsets (200 samples per session, log-normal $\sigma = \ln 2$). The user-study set carries the headline because it has the largest rankable subset and the longest per-session rankings. Multi-knob top-1 stability is ∼ 12 percentage lower than the single-knob result on the same set. Top-3 and top-5 track top-1 on the agent set (78.3% and 75.0%) but drop on the user-study set (61.7% and 59.5%), where longer per-session rankings leave more lower-ranked divergence points available to reorder. The injection top-3/top-5 figures fall sharply because injection rankings have at most 5 items, so one lower-ranked swap can affect both metrics at once.

Clamp-frozen rate and weight-invariance. One possible source of artificial stability is the $[0, 100]$ clamp on the process score. A divergence point's magnitude is $J(\text{alt}_T) - J(\text{user}_T)$, the difference between the alternative's and the user's trajectory-final process scores. A perturbation $W \to W'$ can leave this magnitude unchanged through clamp effects when both terminal scores are already pinned at clamp boundaries in the baseline scoring and remain pinned at the same boundaries after perturbation. The right-hand column of Table 4.1 reports the share of (DP × perturbation) cases in each rankable subset that satisfy this condition: 8.6% on injection, **1.8**% on user-study, and 20.8% on agent. The user-study figure is the smallest of the three, so almost all of the 96.5% top-1 stability on that set reflects genuine perturbation response rather than clamp-driven invariance.

The agent rate (20.8%) is materially higher, and the per-session breakdown explains why. Four of the twenty rankable agent sessions finish with $J(\text{user}_T) = 0$: the S1 sessions across the four Claude agents, where the agent left the build broken at the end of S1 (the same pattern surfaced in §4.3.2, stack 1 Thread 1).

Caveats and scope. These results should be read with three limits in mind. The sensitivity experiment shows that the formula is stable near the production weights, but it does not show that those weights are uniquely correct. A stronger validation would refit the weights against an external outcome dataset, such as longitudinal code-quality measurements, but no such dataset currently exists for this problem. The multi-knob test is also centred on the production weights and only covers the neighbourhood within roughly $\pm 2\sigma$ (about $\times 0.25$ to $\times 4$). Even within that range, the formula degrades on the user-study rankable subset ($\tau = 0.586$ mean), so the robustness claim is local rather than global. Finally, the rank statistics use only the rankable subset of each set ($N = 15/25/20$). The smaller injection subset limits the power of the per-set rank comparison, although the magnitude-recovery statistics in §4.1.2 still use all 45 injection sessions. These limits are treated as threats to validity in §4.2.3 and in the discussion chapter.

4.1.2 Ablation sweep

The sensitivity sweep tests whether rankings survive small weight changes. The ablation sweep tests whether the process-side terms contribute meaningful signal at all. Following the design in §3.6.2, it evaluates all $2^7 = 128$ subsets of the seven process-side weights, giving 5,760 ablation cases per session set. The cleanliness sub-weights are left at their production values because the sensitivity sweep showed that changing them almost never reshuffles the ranking - including them would increase the number of variants from 128 to 8,192 for little additional information. The ablation reports sum-over-sum recovery on all three session sets in parallel (injection 45, user-study 30, agent 48), since magnitudes are defined regardless of how many divergence points a session contains. The cross-set rank comparison in Table 4.7 reports τ on the rankable subsets of injection (15 sessions) and user-study (25 sessions). Sum-over-sum recovery falls to exactly 0.000 on all three sets when every process term is zeroed, and rises to 1.000 under the full formula.

Monotone by active-term count. Table 4.4 groups the per-set ablation cases by the number of active process-score terms. On all three session sets, sum-over-sum recovery increases steadily from 0.000, when every process term is removed, to 1.000 under the full formula. The all-stripped row is the main sanity check: with all process terms set to zero, total magnitude is also zero on every set. This means the reported magnitudes are not coming from a baseline effect independent of the weights. The agent set recovers slightly faster than the injection and user-study sets at low active counts (for example, 0.225 at active-count = 1, compared with 0.170 and 0.190), which is consistent with more of the agent-set magnitude depending on a single term. The per-term breakdown in Table 4.5 shows this directly.

Per-term single-knob recovery. Table 4.5 isolates each process term by keeping that term active and zeroing the other six. The three session types show different patterns. In the injection set, `length` and `broken` recover the most magnitude on their own (0.328 and 0.302), followed by `manualIde` at 0.261 and `lag` at 0.132. In the user-study set, `length` is again the largest single-term contributor (0.426), but `skipTests` (0.371) is larger than `broken` (0.225), which suggests that the more natural user traces expose test-running behaviour more clearly than the controlled injection set. The agent set is different

active terms	injection sum/sum	user-study sum/sum	agent sum/sum	n (per set)
0 (all stripped)	**0.000**	**0.000**	**0.000**	45
1	0.170	0.190	0.225	315
2	0.327	0.353	0.414	945
3	0.473	0.498	0.570	1,575
4	0.612	0.634	0.700	1,575
5	0.745	0.765	0.810	945
6	0.874	0.889	0.909	315
7 (full)	**1.000**	**1.000**	**1.000**	45

Table 4.4: Sum-over-sum magnitude recovery by number of active process-score terms, reported separately for the injection (45 sessions), user-study (30 sessions), and agent (48 sessions) sets. Recovery rises monotonically on all three sets, from exactly 0.000 with every process term removed to 1.000 under the full seven-term formula. The agent set recovers slightly faster at low active counts, consistent with the single-term pattern in Table 4.5.

again: `manualIde` alone recovers the full magnitude budget (1.000), while `skipTests` and `commitGap` contribute exactly 0.000. This matches the pattern in §4.3.2: 41 of the 42 agent divergence points are *IDE-Replay*, which scores entirely through the manual-when-IDE weight W_{MI}. Finally, `gain` alone has sum-over-sum recovery of exactly 0.000 on all three sets because the user and alternative trajectories validate against the same terminal AST, so endpoint cleanliness gain is the same on both sides and cancels in the comparison.

term	injection sum/sum	user-study sum/sum	agent sum/sum
`length`	**0.328**	**0.426**	0.054
`broken`	0.302	0.225	0.405
`manualIde`	0.261	0.170	**1.000**
`lag`	0.132	0.014	0.117
`skipTests`	0.089	0.371	0.000
`commitGap`	0.080	0.125	0.000
`gain`	**0.000**	**0.000**	**0.000**

Table 4.5: Per-term single-knob recovery by session set, with each term active alone and the other six zeroed. `length` is the largest single-term contributor on both the injection and user-study sets, while `skipTests` ranks above `broken` only on user-study. The agent set is dominated by `manualIde` (1.000 alone), consistent with 41 of 42 agent divergence points being IDE-Replay*. `gain` is 0.000 on every set because user and alternative trajectories share the same final AST.*

Per-term leave-one-out recovery. Table 4.6 gives the complementary leave-one-out view: each row removes one term from the full seven-term formula. The results broadly match the single-term analysis in Table 4.5. On the injection set, removing `length` causes the largest loss of magnitude, with sum-over-sum recovery falling to 0.718. The user-study set is similar: removing `length` gives the largest drop (0.636), followed by `skipTests` (0.718). The agent set follows a different pattern. Removing `manualIde` drops recovery from 1.000 to 0.405, whereas removing `length` has little effect (0.950). Removing `skipTests` or `commitGap` has no effect on agent recovery, because those terms carry no agent-set magnitude in this experiment.

The `gain` row behaves differently from the penalty terms. Removing it increases sum-over-sum recovery to 1.060 on injection, 1.120 on user-study, and 1.108 on agent, so the absolute divergence magnitudes become larger than under the production formula. This does not mean that `gain` is a bad term. In these comparisons, the user and alternative trajectories end at the same terminal AST, so endpoint cleanliness gain is shared by both sides rather than separating them. Its main effect is therefore through the $[0, 100]$ score clamp: a shared gain value can move both scores towards the same clamp boundary and

reduce the visible gap created by the penalty terms. Removing `gain` removes some of that compression, so the remaining penalty differences can appear larger. `lag` shows a smaller version of this pattern on user-study (1.015) and agent (1.045), but not on injection (0.842).

removed term	injection sum/sum	user-study sum/sum	agent sum/sum
`length`	**0.718**	**0.636**	0.950
`manualIde`	0.822	0.876	**0.405**
`broken`	0.833	0.960	0.851
`lag`	0.842	1.015	1.045
`skipTests`	0.899	0.718	1.000
`commitGap`	0.943	0.900	1.000
`gain`	**1.060**	**1.120**	**1.108**

Table 4.6: Per-term leave-one-out recovery by session set. Each row keeps six terms active and removes one. `length` has the largest effect on injection and user-study, while `manualIde` is most important on agent. Removing `gain` increases the recovered magnitude above 1.000 because it removes part of the score-clamp compression described in the text.

Cross-set rerun on the user-study sessions. Table 4.7 repeats the leave-one-out test as a rank-stability comparison on the rankable subsets of the injection and user-study sessions (15 and 25 sessions respectively - see §4.1.1). On the user-study subset, `length` is the clearest rank-carrier: removing it gives the lowest τ in the column (0.527). `manualIde` and `broken` follow, while `skipTests`, `commitGap`, and `gain` have more moderate effects. Removing `lag` has the smallest user-study impact ($\tau = 0.784$), which suggests that on these longer human traces it affects magnitude more than the ordering of divergence points. The injection values are lower overall, reflecting the smaller rankable subset, but they still identify `length` and `manualIde` as the strongest rank-carriers.

removed term	injection τ ($n = 15$)	user-study τ ($n = 25$)
`length`	0.500	**0.527**
`manualIde`	0.500	0.588
`broken`	0.192	0.627
`commitGap`	0.374	0.655
`skipTests`	0.444	0.663
`gain`	0.478	0.683
`lag`	0.409	0.784

Table 4.7: Leave-one-out ranking stability on the injection and user-study rankable subsets, sorted by user-study impact. Removing `length` causes the largest user-study drop, while removing `lag` has the smallest effect.

Caveats. Several limits are worth noting. This ablation covers only the seven process-side weights in §3.2.1; the cleanliness sub-weights are excluded because the sensitivity sweep found that they had almost no effect on top-1 stability (§4.1.1). The magnitude tables use the full session sets (45 injection, 30 user-study, 48 agent), since magnitudes are defined for every session. By contrast, the rank-stability table uses only the rankable subsets (15 injection, 25 user-study), as defined in §4.1.1. Finally, the agent set's single-term pattern is driven by the population and detector limits discussed in §4.3.2: W_{MI} recovers all agent magnitude by itself, while `skipTests` and `commitGap` contribute none. This should not be read as a general property of the score formula.

The `gain`-alone row in Table 4.5 is a special case. It stays at the all-stripped baseline (sum/sum 0.000) because the user and alternative trajectories end at the same final AST,

so endpoint cleanliness gain is shared by both sides. In the leave-one-out table, `gain` matters only through its interaction with the $[0, 100]$ score clamp.

4.2 Testing the divergence-point detector

This section evaluates the divergence-point detector against external ground truth. Because the detector is scored against per-kind labels, the first step is to check that those labels are reliable. §4.2.1 reports inter-rater agreement for the protocol described in §3.7. §4.2.2 then reports per-kind precision and recall on the 45 injection sessions, and §4.2.3 discusses a known recall weakness that is left open.

4.2.1 Inter-rater reliability

Table 4.8 reports per-kind Cohen's κ for the three rater pairs on the 45-session set, using the labelling protocol and interpretation thresholds from §3.7. All four divergence kinds reach at least substantial agreement. Ordering and IDE-replay are unanimous across all three labellers ($\kappa = 1.00$ for every pair). Rework is also high, with $\kappa = 0.86$ between the author and each independent rater, and $\kappa = 1.00$ between P1 and P2. Hygiene is the lowest but still substantial, ranging from $\kappa = 0.72$ to $\kappa = 0.86$.

kind	Author vs P1	Author vs P2	P1 vs P2
Ordering	1.000	1.000	1.000
IDE-Replay	1.000	1.000	1.000
Rework	0.861	0.861	1.000
Hygiene	0.848	0.723	0.862

Table 4.8: Per-kind Cohen's κ on the 45-session set for all three rater pairs. Ordering and IDE-replay are unanimous; rework and hygiene both reach at least substantial agreement.

The remaining disagreements are concentrated in a small number of sessions. For Rework, only two of the 45 sessions differ: both independent raters label session 024 (borderline suboptimal ordering) as Rework while the author does not, and the author labels session 043 (borderline add-then-revert) as Rework while neither rater does. Since P1 and P2 agree with each other in both cases, the author's labels on these sessions may be the outliers. Hygiene disagreements are also localised. P2 labels three manual-move-method sessions (014, 015, 016) as Hygiene while the author and P1 do not, and the author labels session 039 (borderline no-commit stretch) as Hygiene while neither rater does. Following the protocol in §3.7, these disagreements are reported as findings rather than resolved by editing the labels.

4.2.2 Divergence experiment

The third experiment evaluates the divergence-point detector against ground truth, using the protocol defined in §3.7.6. Precision is perfect for all four kinds: Rework, Hygiene, Ordering, and IDE-Replay all score 1.00. Across the 45 injections, the detector surfaces an alternative in 36 cases. Of those, 26 have positive magnitude, meaning the alternative strictly beats the user's trajectory under the production formula. Every positive-magnitude catch is ranked top-1 within its session.

Precision and recall. Table 4.9 reports per-kind precision and recall. Rework and Hygiene are perfect on both axes: all 9 Rework injections and all 13 Hygiene injections are caught with 1.00 precision. IDE-Replay is also precision-perfect. Ordering is different: precision remains 1.00, but recall is 0.36 on any-expected and 0.40 on injection-only. This

is because the reorder synthesiser's `splitOnInvalid` validator check (§3.3.2) rejects any window containing a step the synthesiser cannot reproduce, and most Ordering-injection windows in this set contain at least one such step. This recall gap is documented and its rationale discussed in §4.2.3. The reliability of the ground-truth labels is reported in §4.2.1.

kind	precision	recall (injection)	recall (any expected)
Ordering	1.00	0.40	0.36
IDE-Replay	1.00	0.76	0.76
Rework	1.00	1.00	1.00
Hygiene	1.00	1.00	1.00

Table 4.9: Per-kind precision and recall on the 45-session set. The "injection" recall column counts only the kind the playbook explicitly injected; "any expected" counts any kind the session's `expected_kinds` label includes.

Detection quality. Precision and recall show whether the detector finds the expected kinds, but they do not show whether the proposed alternatives are actually better than the user's trajectory. Table 4.10 therefore looks at the quality of the emitted divergence points. For each kind, it reports how many divergence points are emitted and whether the best alternative beats, ties, or loses to the user's trajectory under the production score formula. Across all kinds, best-alternative magnitudes range from -2 to $+23$, with a mean of $+6.07$.

Hygiene is the strongest case: all 13 Hygiene divergence points strictly improve on the user's trajectory. IDE-Replay also performs well, with 12 improvements from 16 divergence points (75.0%). Of the four non-improvements, three are tied at zero because the user's score is already pinned at the bottom of the $[0, 100]$ range, so the IDE-driven alternative cannot score lower either. The remaining case is a genuine loss, where the IDE alternative scores 5 points below the user's manual edit. These misses mostly come from the score floor: in three cases the user's score was already clipped at the bottom of the $[0, 100]$ range, so the IDE alternative could not show an improvement even if it avoided the manual-IDE penalty.

Rework is more mixed. It beats the user in 6 of 13 divergence points (46.2%), ties once, and loses six times. This is partly because the rework synthesiser is deliberately simple: it removes add-then-undo code chunks without modelling why the intermediate states existed. In practice, a user may add a guard, run tests, decide it is unnecessary, and remove it. The final diff looks like wasted motion, but the temporary code may have kept later intermediate states compiling or passing tests until the rest of the refactor caught up. Removing it just because it disappears later can therefore introduce extra broken-build checkpoints in the alternative path. This is also a methodological point: undoing work is not always bad. A slightly longer path that verifies intermediate correctness can score higher than the shortest path to the same final state. The Rework beat rate therefore separates cases where the removed code chunk really was wasted motion from cases where it helped preserve a better intermediate trajectory, while also showing the limit of a simple chunk-stripping approach.

Ordering emits 24 divergence points after considering 194 valid permutations. Each divergence point corresponds to one reorder window, and its magnitude is taken from the best permutation found for that window. Ten windows (41.7%) beat the user's order, 12 tie at zero, and 2 are worse than the user's order. The important point is that these alternatives must have an identical final AST as the user's path, so endpoint terms alone cannot separate them. The production formula separates them through W_{LAG}, which rewards paths that improve intermediate cleanliness earlier. In the lag ablation, only 4 Ordering windows

remain positive, and the two negative cases collapse to zero.

kind	DPs	beats user	ties user	loses to user	beats fraction	max magnitude
Rework	13	6	1	6	**46.2%**	8 points
Hygiene	13	13	0	0	**100%**	9 points
IDE-Replay	16	12	3	1	**75.0%**	23 points
Ordering	24	10	12	2	**41.7%**	9 points

Table 4.10: Per-kind detection quality. "DPs" gives the number of emitted divergence points. The beats, ties, and loses columns group those points by the sign of $J(alt_T) - J(user_T)$ under the production formula. Hygiene improves on the user in every case; Ordering has many ties because several reordered alternatives reach the same final state with the same final score.

Negative Ordering magnitudes. Two of the 24 Ordering windows have negative magnitude under the production formula, meaning the user's original order scores better than the synthesised alternative. Session 043 is the clearest example: both paths reach the same final state, but the alternative has worse intermediate cleanliness, so it accumulates more lag penalty. This shows that W_{LAG} is not just increasing positive Ordering magnitudes; it also allows the formula to recognise cases where the user's order was the better one.

Injection prominence. Table 4.11 reports how prominently the injected problem appears in the detector output. The detector catches 26 injections with a positive-magnitude alternative: 8 Hygiene, 12 IDE-Replay, 5 Rework, and 1 Ordering. In every one of these cases, the injected behaviour is the highest-magnitude divergence point in its session, giving a mean rank of 1.00 across all four kinds. A user who opens only the top dashboard recommendation in §3.5 would therefore see the deliberately injected problem in every positive-magnitude catch.

This result is partly a consequence of the controlled-injection design. Each playbook session contains one deliberately introduced problem, so that problem is likely to dominate the session's ranking. In uncontrolled real-world sessions, several issues may be present at once, and the top-ranked item may not correspond to one clear expected kind. We therefore do not report a prominence number for the participant sessions in §4.3.1. Instead, we report their per-session, per-kind distribution.

injected kind	n caught	@ rank 1	mean rank
Hygiene	8	8	1.00
IDE-Replay	12	12	1.00
Ordering	1	1	1.00
Rework	5	5	1.00

Table 4.11: Injection prominence: rank of the caught injection's divergence point within its session's magnitude ordering. Every positive-magnitude catch is ranked top-1 across all four kinds.

Caveats. A caveat worth noting when interpreting these divergence numbers is that the dataset contains 45 hand-recorded sessions, so some categories have few examples: for example, there are only 5 ordering injections. The ratios are therefore best read as directional evidence, not as estimates with tight confidence intervals, and raw counts are reported alongside percentages for this reason.

4.2.3 Known limitation: ordering recall

Ordering has the weakest recall of the four divergence kinds: 0.36 on any-expected and 0.40 on injection-only. This mainly comes from the `splitOnInvalid` validator check in §3.3.2.

If a reorder window contains a step that the synthesiser cannot reproduce, the window is split into single-step windows and no permutations are generated.

This matters because ordering often changes process score under the production formula. Commuting refactorings may reach the same end state, but their intermediate cleanliness trajectories can still differ. The lag term W_{LAG} assigns different magnitudes to those trajectories: Table 4.10 shows that 10 of 24 admitted ordering windows have positive magnitude under the production formula, compared with only 4 when W_{LAG} is removed. In other words, windows currently excluded by `splitOnInvalid` cannot be assumed to have zero magnitude; some would likely become positive findings if the synthesiser supported more step kinds. Closing this gap therefore depends on extending the synthesiser engine, as discussed as future work in Chapter 5. Until then, the ordering recall should be read as a lower bound on what the detector could surface with a more complete engine, rather than as a limitation of the score formula itself.

4.3 User and agent studies

§4.1 and §4.2 test the score formula and the detector in isolation. Here, the focus shifts to the tool in use: whether participants change how they refactor after seeing dashboard feedback. The feedback consists of divergence points ranked by score-delta magnitude, each paired with a concrete alternative trajectory.

§4.3.1 reports the main end-to-end study: 30 sessions with five human participants on a new codebase, split between a feedback group (three participants) and a no-feedback group (two participants). §4.3.2 then reports a 48-session agent extension on the same codebase, covering eight agent/task combinations. This second study is included to see how far the same tooling carries over to agent traces - the section also explains where this tool's human-session assumptions break down in that setting.

4.3.1 User study

The 30-session user study moves beyond the controlled-injection setting by applying the detector to sessions recorded by independent participants on a different codebase, without any pre-planted divergence kinds. The study is mainly descriptive: it checks whether the surfaced divergence points still look coherent in this less controlled setting, and whether their frequency changes as participants complete repeated sessions. Because some participants saw dashboard feedback between sessions and others did not, it also gives a comparison between changes associated with feedback and changes that may simply come from repeating the tasks.

Five participants, referred to throughout as P1 through P5, were recruited from the MEng Computing cohort at Imperial College London. Each participant completed six recorded refactoring sessions on the same order-processing codebase (`user-study-fixture/`), which is structurally separate from the library codebase used in §4.2.2. This gives 30 sessions in total. P1, P2, and P3 formed the feedback group: after each session, they viewed the dashboard's divergence points and trajectory advice before starting the next session. P4 and P5 formed the no-feedback group: they followed the same six-session playbook on the same codebase, but did not see the dashboard between sessions. The playbook prompts, codebase, plugin instrumentation, and reset script were otherwise identical across groups. Two participants, P1 and P2, also independently labelled the 45-session injection set before starting their own user-study sessions. That labelling is used in the three-rater inter-rater agreement reported in §4.2.1, so it is not repeated here.

Group assignment was randomised before the sessions began: two of the five participants were drawn uniformly at random to form the no-feedback group. The resulting group

sizes ($n = 3$ vs $n = 2$) are too small for hypothesis testing, so this section does not report a p-value. The comparison is instead directional: whether the gain-stripped process score and the per-session divergence-point counts move in the same direction in both groups, or whether the feedback and no-feedback groups separate.

Descriptive findings on the participants' sessions. For each session, the playbook gave participants a concrete refactoring task, including the relevant files, methods, and intended behaviour-preserving change. It did not specify which IntelliJ actions to use, when to test or commit, or which divergence kinds the detector might report. Since the playbook deliberately avoided pre-declared `expected_kinds`, these sessions cannot support the same precision/recall analysis used in §4.2.2. Instead, they support a descriptive view of what the detector surfaced to the users: Table 4.12 reports the per-kind distribution and per-session counts, grouped by study group and participant.

group	participant	*IDE-Replay*	*Rework*	*Hygiene*	*Ordering*	total	DP / ses
with-feedback	P1	12	2	5	2	21	
with-feedback	P2	9	0	8	1	18	
with-feedback	P3	5	0	5	0	10	
with-feedback (sub)	$n = 18$ **sessions**	**26**	**2**	**18**	**3**	**49**	
baseline	P4	21	3	16	0	40	
baseline	P5	13	0	17	2	32	
baseline (sub)	$n = 12$ **sessions**	**34**	**3**	**33**	**2**	**72**	
combined	$n = 30$ **sessions**	**60**	**5**	**51**	**5**	**121**	

Table 4.12: Per-participant divergence-point counts across the 30-session user study, grouped by feedback condition. The table reports aggregate per-kind totals; full per-session breakdowns are available in the reproducibility notebook (`tool/fixtures/notebooks/experiments.ipynb`).

Across the 121 divergence points, IDE-replay accounts for 49.6% and hygiene for 42.1%, while rework and ordering each account for 4.1%. This is broadly in line with the 45-session injection set in §4.2.2, where IDE-replay and hygiene also dominate. Both study groups still exercise all four kinds, but rework and ordering appear only occasionally.

The clearest comparison in the table is the per-session divergence-point rate: 2.7 DPs per session in the feedback group, compared with 6.0 in the no-feedback group, a 2.2× difference. The split is not just a constant offset between groups. As the trajectory table below shows, both groups start with similar totals at S1 and then separate across the remaining 5 sessions.

All five participants registered 0 divergence points on S04, where the task was to remove duplicated validation logic by keeping a single shared version of the checks. All five completed it through direct manual edits. There was no rework, skipped-test pattern, IDE-replayable manual sequence, or reorder window for the detector to report. This suggests that the detector was not simply producing findings on every session: for this relatively clean manual task, it produced none.

In the post-study interviews, both feedback participants from the original cohort identified IDE-replay as the most actionable kind. One noted that they "didn't realise that IntelliJ could do all of these refactorings", which helps explain why they had not used them earlier. Both identified ordering as the least actionable kind, with one asking "how can I actually apply that in the future without knowing beforehand".

Behavioural trajectory across the six-session arc. We next look at how divergence-point counts change as participants complete more sessions, and whether the same pattern appears in the no-feedback group. Table 4.13 aggregates the counts session by session within each group. In the feedback group, the main pattern is a downward trend after S2:

the hygiene-driven peak in S2 is followed by lower counts in each later session except S5. By the final session the aggregate has fallen to 2 divergence points, compared with 12 in S1. The no-feedback group does not show the same decline: it starts at a similar level (10 in S1) and remains higher through the later sessions, ending at 15 in S6.

The groups separate most clearly from S3 onwards. The feedback group stays on a mostly downward path after S3, while the no-feedback group remains in the low-to-high teens through the same part of the study. This matches what feedback-group participants reported in their interviews: they described running tests after each change, committing more often, using the provided automated tools, and using IDE refactorings more frequently.

group	kind	S1	S2	S3	S4	S5	S6	Δ
with-feedback	*IDE-Replay*	8	6	6	0	5	1	−7
with-feedback	*Hygiene*	3	11	2	0	2	0	−3
with-feedback	*Rework*	0	0	2	0	0	0	0
with-feedback	*Ordering*	1	0	1	0	0	1	0
with-feedback (sub)	**total**	**12**	**17**	**11**	**0**	**7**	**2**	**−10**
baseline	*IDE-Replay*	5	3	11	0	7	8	+3
baseline	*Hygiene*	4	11	7	0	6	5	+1
baseline	*Rework*	0	1	1	0	0	1	+1
baseline	*Ordering*	1	0	0	0	0	1	0
baseline (sub)	**total**	**10**	**15**	**19**	**0**	**13**	**15**	**+5**

Table 4.13: Per-kind divergence-point trajectory across the six-session arc, summed within each group. The feedback aggregate trends downward after S2 and finishes at 2 divergence points (Δ = −10), while the no-feedback aggregate ends higher than it started (Δ = +5). Both groups register 0 divergence points on S04. IDE-replay and hygiene carry most of the group-level signal.

The divergence between the two groups is mostly carried by IDE-replay and hygiene. In the feedback group, IDE-replay falls from 8 to 1 and hygiene from 3 to 0. The S2 hygiene spike is consistent with participants first seeing commit-cadence feedback after S1 and then changing their behaviour from S3 onwards. In the no-feedback group, neither kind shows the same drop: IDE-replay ends +3 across the arc, and hygiene ends +1 rather than −3.

Rework and ordering play a smaller role. Ordering accounts for only 5 positive-magnitude divergence points across all 30 participant sessions. Rework is also concentrated in a few harder sessions, where a short run of undo/redo actions can produce several flags. One feedback-group participant raised this as a metric-design concern in interview, saying that rework "picked up on undos + redos a lot (in a bad way) [-] over penalised" sequences caused by mistyped keys. The per-session detail shows the same pattern.

The six sessions are not equally difficult. S2, which focuses on magic-number renames, and S4, which removes duplicated validation logic, are short mechanical tasks where divergence-point counts are low for both groups. S3 and S6 are more open-ended, and the playbook gives more room for hygiene and rework flags to appear. For that reason, the raw counts are most useful when comparing the two groups within the same session sequence, rather than as a direct measure of improvement from one task to another. The no-feedback group helps with this interpretation: both groups see the same S2 hygiene rise and the same quiet S4 session, so the later separation between them is less likely to be explained by the task sequence alone. The same caveat applies to the agent extension in §4.3.2.

Process scores under the production and gain-stripped weightings. The process score is also affected by task difficulty, because some prompts allow more cleanliness improvement than others. To separate this from the way participants carried out the

work, we inspect the score in two forms: the production weighting, and a *gain-stripped* weighting where both the cleanliness-gain weight W_g and the intermediate-cleanliness lag weight W_{LAG} are set to zero. This second version is closer to a measure of process discipline: whether the participant kept the project working, tested and committed at sensible points, used available IDE refactorings, and avoided unnecessary extra steps.

The stripped score is used only for this analysis. In the main evaluation, removing W_g would be inappropriate because the production formula is meant to reward real code improvement, not just a tidy sequence of actions. The user-study setting is narrower: the playbook fixes the structural change each participant is asked to make, so it is useful to temporarily remove the two terms tied directly to the cleanliness scalar C. These are W_g, which rewards endpoint cleanliness, and W_{LAG}, which rewards reaching cleaner intermediate states earlier. What remains are the five terms that do not depend on C: broken-time, skipped-tests, manual-when-IDE, length-bonus, and commit-gap. In this view, the score asks how disciplined the participant's process was, rather than how much cleaner the final code became. Table 4.14 reports this gain-stripped score across the six-session arc for all five participants, grouped by whether they saw feedback between sessions.

group	participant	S1	S2	S3	S4	S5	S6	Δ	slope
with-feedback	P1	25	28	42	47	40	50	+25	+5.0
with-feedback	P2	16	18	31	47	38	47	+31	+6.2
with-feedback	P3	39	20	43	49	32	50	+11	+2.2
with-feedback (mean)	($n = 3$)	**26.7**	**22.0**	**38.7**	**47.7**	**36.7**	**49.0**	**+22.3**	**+4.47**
baseline	P4	22	18	10	33	16	18	−4	−0.8
baseline	P5	23	17	12	45	17	23	0	0.0
baseline (mean)	($n = 2$)	**22.5**	**17.5**	**11.0**	**39.0**	**16.5**	**20.5**	**−2.0**	**−0.40**

Table 4.14: Trajectory-final process score across the six-session arc for each participant under the gain-stripped weighting ($W_g = 0$, $W_{\text{LAG}} = 0$), grouped by whether participants saw feedback between sessions. The production-weighted view is dominated by the cleanliness-gain term and is reported per-session in the reproducibility notebook. Δ is $S6 - S1$ and slope is $\Delta/5$ in score units per session. The group means aggregate across participants within each group.

Two patterns are worth noting. First, all three participants who saw feedback finish above where they started (+25, +31, +11). The two participants who did not see feedback do not show the same improvement: P4 finishes 4 points below their starting score, and P5 ends exactly where they started. The group means also move in different directions: +22.3 vs −2.0, or +4.47 per session vs −0.40 per session. Given the small group sizes ($n = 3$ and $n = 2$), this is treated as a directional result rather than a hypothesis test.

Second, the session-by-session pattern is still shaped by the tasks themselves. Under the stripped score, S2 dips for three of the five participants, S4 is high for almost everyone, and the separation between the two groups starts to build from S3 onwards. This fits the structure of the playbook. S2, which focuses on magic-number renames, and S3, which consolidates duplication within a class, are the first sessions where the dashboard's hygiene and IDE-replay advice could have affected participants who saw feedback after S1. The largest single-session gap appears at S3 (38.7 vs 11.0). By contrast, S4 is a cleaner execution task for both groups, so the scores move closer together again (47.7 vs 39.0).

The playbook does not get easier over time, although the difficulty is not monotonic: S2 and S4 are shorter mechanical tasks, while S3 and S6 are more open-ended. This means the improvement among participants who saw feedback is not explained by later tasks being easier. The two participants who did not see feedback follow the same task order but stay flat or decline. P3's smaller slope (+2.2) is also unsurprising: they already start at 39 in S1, closer to the ceiling than P1 or P2, so there is less room to improve.

Feedback vs no-feedback group. The group-level comparison is the clearest evidence in this chapter that the tool may have affected participant behaviour. Group assignment was randomised within the same participant pool, and the codebase, playbook order, tasks, and plugin instrumentation were the same for both groups. The main difference was whether participants saw the dashboard between sessions. The group means in Table 4.14 show a +22.3 vs −2.0 gain-stripped Δ across the arc (a 24.3-point gap, or +4.47 vs −0.40 per session), while the group totals in Table 4.13 show a −10 vs +5 divergence-point Δ (a 15-DP gap). Both measures point in the same direction.

The gain-stripped score is less affected by how well a participant knows the codebase, because its five active terms are commit cadence, skipped tests, manual-when-IDE rate, broken-build time, and the length bonus. None of these directly rewards knowing the domain model or finding a cleaner final design. The design still cannot fully separate dashboard feedback from two related effects: participants may learn what the dashboard rewards and adapt to it, or they may simply become more comfortable with the style of tasks over time. Both are discussed in the threats-to-validity chapter (§5.2.3). The no-feedback group does rule out one simpler explanation: the improvement is unlikely to be caused only by later tasks being easier, because the participants who did not see feedback followed the same playbook in the same order and did not improve. The main remaining limit is sample size. With $n = 3$ vs $n = 2$, the study is too small for a hypothesis test, so the result is reported only as a directional finding: under random group assignment, the participants who saw feedback improved, while those who did not see feedback stayed flat or declined. No p-value or confidence interval is claimed.

One qualitative finding is not visible from the trajectory tables but is important for future tuning. The hygiene signal can fire when a participant is pausing to think, rather than when they have actually moved on without checking their work. One participant who saw feedback described this as being "penalised for not running tests when wasn't quite finished but just was thinking", and the S2 hygiene peak in both groups shows the same pattern. This issue, like the rework flags caused by undo/redo sequences, is better treated as a metric-tuning problem than a detector bug. The detector is reporting patterns that are present in the trace, but the current metric sometimes treats legitimate pauses or mistyped-key recovery more harshly than a participant would.

Per-step inspection as a usability win. Both interviewed participants picked out the dashboard's per-checkpoint detail panel as useful, separately from the four detector kinds. One said it made it easy to "see what action you made to cause the issues to happen", and described the trajectory graph showing the best reachable score from each point as "egging you on to be a better person". This suggests that the alternative-trajectory overlays were read as constructive rather than punitive. The other participant said they could "see the specific point you went wrong", liked the cleanliness-signal "breakdowns" on each step, and described the dashboard as "easy to track session at specific points and code smells". This supports the design choice in §3.2.6: exposing per-step smell attribution and metric shifts was not just an implementation detail, but part of what participants found useful.

4.3.2 Agent extension experiment

Framing. This section reports an extension experiment rather than a second full evaluation. The tool in this thesis was designed around human edit traces: it records IDE refactor-menu actions, groups edit bursts after a 2-second idle, and measures commit cadence using the 60-second composite window from Murphy-Hill et al. [5]. These signals do not transfer cleanly to coding agents, which emit file changes in a different shape (§4.3.2 itemises the main biases this creates). For that reason, the agent runs are used as a stress

test for the detectors, not as a claim that the tool is a general way to evaluate coding agents.

Setup. Eight agents ran the same six-session playbook as the human participants, on the same `user-study-fixture/` codebase, giving 48 sessions in total. The runs were carried out on 2026-05-28, so the results should be read as a snapshot of those model and harness versions at that time. The agents cover three model families (Claude, OpenAI GPT, Gemini) and three command-line harnesses (Claude Code, OpenCode, Cursor CLI). Two agents were run without feedback, mirroring the baseline group in the human study; the other six received feedback between sessions.

For the feedback agents, we pasted the markdown output of the `feedbackForAgent` script (`tool/analysis/.../experiment/FeedbackForAgent.kt`) into the next session's prompt. This script converts the dashboard's divergence points and trajectory advice into a plain-text summary, so the agent received the same information as a human participant, but as text rather than through the rendered dashboard. The two no-feedback agents were Claude 4.7 Opus Medium Thinking running through Claude Code and GPT-5.5 Medium Thinking running through OpenCode; both ran the same playbook on the same codebase but were not given dashboard feedback.

Each agent followed the same lightweight setup so the trace could be captured consistently. Tests were run through a wrapper script that marked the start and end of each Gradle test run so the plugin could detect them. Apart from starting and stopping the recording at session boundaries, and pasting feedback into the next prompt, there was no human intervention during a session.

Agent-level results. Table 4.15 reports the gain-stripped final process score for each agent across the six sessions, together with the total number of divergence points it produced. The gain-stripped score is used for the same reason as in the human user study: it focuses on process discipline rather than cleanliness gain, which is more tied to the requested structural change than to how the agent carried out the work.

#	agent	group	S1	S2	S3	S4	S5	S6
1	Claude Opus 4.7 with Claude Code	feedback	35	40	40	50	42	37
2	Claude Opus 4.7 Medium Thinking with OpenCode	feedback	37	40	40	50	42	40
3	Claude Opus 4.7 Medium Thinking with Cursor CLI	feedback	35	40	40	50	42	41
4	Gemini Flash 3.5 Medium Thinking with OpenCode	feedback	40	40	40	50	42	38
5	GPT-5.5 Medium Thinking with Cursor CLI	feedback	32	40	40	50	42	42
6	GPT-5.5 Medium Thinking with OpenCode	feedback	41	40	40	50	42	40
7	Claude Opus 4.7 Medium Thikning with Claude Code	baseline	36	40	40	50	42	41
8	GPT-5.5 Medium Thinking with OpenCode	baseline	33	40	40	50	42	41
with-feedback (mean, $n = 6$)			**36.7**	**40.0**	**40.0**	**50.0**	**42.0**	**39.7**
baseline (mean, $n = 2$)			**34.5**	**40.0**	**40.0**	**50.0**	**42.0**	**41.0**

Table 4.15: Per-agent gain-stripped trajectory and total divergence-point count across the eight agents. Agents 1–6 received feedback between sessions; agents 7 and 8 are no-feedback baselines. The group means aggregate across agents within each group.

The table shows the same easy-session pattern seen in the human study: every agent scores $J = 50$ at S4 and $J = 42$ at S5. Six of the eight agents finish above where they started, while Gemini with OpenCode finishes at -2 and GPT-5.5 with OpenCode finishes at -1. The largest improvement is $+10$ for agent 5.

The divergence-point counts are also small. Across all 48 agent sessions there are 42 divergence points: 41 IDE-replay, 1 ordering, and none for rework or hygiene. This gives a much lower rate than in the human study: 0.7–1.5 per session for the feedback agents

and 0.5–0.7 for the no-feedback agents, compared with 2.7–6.0 per session for the human participants. In this setup, the agents start from a much higher procedural-discipline baseline.

Feedback vs no-feedback group. The group-level comparison in Table 4.15 does not mirror the human study. The six agents that received feedback gain +3.0 on average, while the two no-feedback agents gain +6.5 on average. Both groups improve, but the no-feedback agents improve more (+1.3 per session compared with +0.6). The divergence-point totals point the same way: feedback agents produce 36 DPs across 36 sessions (1.0 per session), while no-feedback agents produce 7 DPs across 12 sessions (0.6 per session). In these agent sessions, between-session feedback does not appear to improve agent process scores.

Several factors help explain this result without implying that the feedback harmed the agents:

- **Less room to improve.** Agents already start with much higher gain-stripped scores than the human no-feedback group (mean S1 $J \approx 37$ across agents vs ≈ 22.5 for humans). Their no-feedback divergence-point rate is also far lower (≈ 0.5 per session vs ≈ 6.0 per session), so there is less space for feedback to move the measured score.
- **Detector coverage.** Three of the four divergence-point kinds rarely fire on agent traces at all (see §4.3.2). If an agent changes its behaviour in ways those detector kinds do not capture, the score will not show it.
- **Small no-feedback group.** The no-feedback group has only two agents. GPT-5.5 Medium with OpenCode starts at $S1 = 33$, lower than the other agents, and then gains +8. Because there are only two no-feedback agents, that one low starting point has a large effect on the group mean.

Cross-agent effects. The six feedback agents also give a small descriptive comparison across model and harness choices. There is only one run for each combination, so the comparison is only illustrative. Holding OpenCode fixed, Claude is the only model with a positive slope (+0.60), while Gemini and GPT finish slightly below where they started (-0.40 and -0.20). Gemini also produces the most divergence points in this slice (9, compared with 4 for Claude and 5 for GPT). Holding the Claude model fixed, Cursor CLI has the steepest improvement (+1.20), followed by OpenCode (+0.60) and Claude Code (+0.40). The remaining comparison, Claude with Claude Code versus GPT with OpenCode, is close in divergence-point count (5 each), but Claude with Claude Code has the higher slope (+0.40 vs -0.20).

Limits of applying the detector to agents. The agent traces show three places where assumptions from human IDE sessions break down:

- **No IDE refactoring action.** Agents edit files through text-based tool calls rather than IntelliJ's Refactor menu. As a result, structural refactorings by agents are seen as manual edits, so the *IDE-Replay* detector fires when an IntelliJ refactoring could have produced the same change. This accounts for 41 of the 42 agent divergence points. The feedback is meaningful, but the agent cannot act on it in the same way a human IntelliJ user can.
- **Fewer commit-gap opportunities.** The *Hygiene* detector reports a commit gap after six refactoring checkpoints without a commit. Agents tend to batch many file

changes into fewer checkpoints, so the threshold is rarely reached. This matches the 0 hygiene divergence points across all 48 agent sessions.

- **Agent edits arrive in bursts.** The edit-burst tracker groups file writes that occur close together. This still catches tool-applied file writes, but an agent often emits several edits in quick succession and then pauses to reason. That collapses much of a turn into one burst, weakening the signal used by the *Rework* and *Ordering* detectors. This is consistent with the 0 rework divergence points and only 1 ordering divergence point across the agent sessions.

These limits do not change the main human-trace evaluation, but they do affect how the agent results should be read. Table 4.15 is therefore best treated as descriptive evidence about applying the tool outside its intended setting, rather than as a verdict on the tool's use for human-developer refactoring.

Agents articulate adjustments in writing, but the score does not reflect them. Although the aggregate ΔJ does not show a feedback effect, the transcripts show that the agents receiving feedback do read it and planned changes for the next session. Five of the six agents write explicit statements about what they will change next, and those statements line up with warnings from the previous session. The clearest examples are:

- Gemini 3.5 Flash with OpenCode, after S03, responded to a `REFACTORING_INTRODUCED_SMELLS` warning by planning "Atomic and Direct Extractions" and saying it would avoid intermediate states that were incomplete or introduced smells.

- Claude Opus 4.7 Medium with OpenCode, after S01 warnings about broken builds, test regressions, and process-score degradation, planned to make renames atomically: "change the declaration and all call sites in a single edit cycle before running tests/committing".

- OpenAI GPT-5.5 Medium with OpenCode, also after S01, said it would identify affected symbols and call sites first, then perform each rename as "one coherent change" before moving to the next.

- Claude Opus 4.7 Medium with Cursor CLI, after two S05 IDE-replay flags, noted that the analyser scores code shape and planned to inline throwaway intermediates when a temporary variable was used only once.

The exception is Claude 2 Opus 4.7 with Claude Code. It gives only short acknowledgements, such as "Acknowledged - session 1 feedback received. Waiting for the session 2 prompt.", but its behaviour still changes. In S01 it made one bulk commit after all three renames; in S02 it moved to "one commit per step" with `./agent-test` between steps, even though its pre-S02 message did not describe that change.

The main point is therefore not that only some agents engaged with the feedback. At the transcript level, most of the agents that received feedback appeared to read it and use it when planning the next session, with Claude 2 Opus 4.7 with Claude Code simply being less explicit in writing. What does not appear is a matching separation in ΔJ. This is likely because many of the written adjustments concern broken-build time and atomic commit cadence, while the measured agent sessions are dominated by IDE-replay divergence points (41 of 42 DPs). As discussed in §4.3.2, that detector is not very sensitive to the kinds of process changes agents can actually make.

4.4 Summary

This chapter reports four experiments. The first two test the machinery of the approach. §4.1 shows that divergence-point rankings are locally stable under plausibly sized weight perturbations, and that each of the seven process-side terms contributes measurable magnitude, although the important terms differ across the injection, user-study, and agent sessions. §4.2 then evaluates the detector against three-rater ground truth on the 45 injection sessions. Per-kind precision is 1.00 across all four kinds at Cohen's $\kappa \geq 0.72$, and every positive-magnitude catch is the top-ranked divergence point in its session, with one known recall limit on *Ordering* documented in §4.2.3.

The second pair of experiments studies the tool in use. In the 30-session randomised-group user study, participants who saw feedback gained +22.3 on the gain-stripped process score and reduced total divergence points from 12 to 2. Participants who did not see feedback lost 2.0 and rose from 10 to 15 divergence points. Since the groups are small ($n = 3$ vs $n = 2$), this is treated as a directional result rather than a hypothesis test. §4.3.2 extends the same protocol to eight agents as a feasibility check. The human-study pattern does not transfer cleanly to agents: agents receiving feedback often write down sensible adjustments, but those adjustments do not show up clearly in the score, which is consistent with the detector's mismatch to agent edit patterns.

The next chapter discusses the main threats to validity behind these results. The conclusion then draws together what the experiments support, what remains uncertain, and where the work could be taken next.

Chapter 5

Discussion

This chapter brings together the main results of the project. It first summarises what was built, then discusses the main limitations, states the headline findings, and closes with directions for future work.

5.1 Summary of contributions

The project makes two main contributions. The first is the process-quality score J. This combines the endpoint cleanliness gain with five process penalties – broken builds, skipped tests, manual edits where an IDE refactoring was available, long gaps between commits, and an intermediate-cleanliness lag penalty – plus a step-savings bonus for alternatives that reach the same end state in fewer steps. The second contribution is the divergence-point detector. It finds points in a refactoring session where a better alternative exists, assigns them to one of four kinds (*IDE-Replay*, *Rework*, *Hygiene*, *Ordering*), and produces a concrete alternative trajectory for each. These alternatives come from four synthesisers: one for reordering steps, one for replaying manual edits as IDE refactorings, one for removing self-cancelling rework, and one for modelling different test and commit choices.

The work is supported by a 45-session hand-recorded dataset on a purpose-built Java fixture, with multi-label ground-truth annotations, a 30-session randomised-group user study on a separate fixture; and an 8 agent, 48-session extension run on the original fixture. It also includes the IntelliJ plugin used to capture event traces, the JCEF dashboard used to surface divergence points, and a reproducible notebook from which every number in Chapter 4 can be regenerated.

5.2 Limitations

This section sets out the main limitations to keep in mind when reading the results in Chapter 4 and the headline findings in §5.3. Some mitigations have already been described alongside the relevant design choices, including the three-way inter-rater check, deterministic ranking tie-breaks, the multi-knob sensitivity analysis, and the reproducible notebook used for the reported numbers. They are only repeated here where they directly affect the limitation being discussed.

5.2.1 Construct and weight calibration

There is no external ground truth for "refactoring quality". The process-quality score J measures signals that suggest a weaker refactoring process: broken builds, skipped tests, manual edits where an IDE refactoring would have worked, and long gaps between commits. It should not be read as a predictor of later bug rates, time-to-merge, or other

downstream outcomes. There is no dataset linking session-level process signals to maintenance outcomes at this granularity, so the weights are based on the severity orderings described in §3.2.2, rather than fitted to outcome data.

The weights are plausible, but not uniquely calibrated. The production weights follow the severity ordering described in the methodology, but the exact values are still a design choice, and other values within the same ordering could also be justified. The sensitivity analysis in §4.1.1 gives some reassurance: on the rankable user-study subset (25 of 30 sessions), the top-ranked divergence point stays the same in 96.5% of single-knob perturbations and 84.6% of multi-knob Monte Carlo runs. This suggests that the formula is not fragile near the chosen weights, but it is still only a local robustness result, not a claim about arbitrary reweighting.

Agreement was checked on kinds, not ranking. The Cohen's κ result in §4.2.1 shows that three independent labellers usually agree on which divergence kinds appear in each session. What it does not show is whether they would rank those divergence points in the same order as the score, even though the dashboard presents the highest-ranked points first. That would require a separate ranking study, for example comparing human rankings with the production ranking using Kendall τ-b. This is left as future work in §5.4.

5.2.2 Study design and statistical power

The user study is randomised, but too small for hypothesis testing. Five participants completed six sessions each, giving 30 sessions in total. The study groups were assigned uniformly at random before the study began, but the groups are still very small ($n = 3$ vs $n = 2$). Variation within each group is about the same size as the difference between groups, so the results chapter reports only the direction of the effect, without a p-value or confidence interval. The no-feedback group used the same playbook, and its downward trend makes it less likely that the tasks simply became easier over time. However, two explanations remain: participants with feedback may have learned what the dashboard rewards, or they may simply have become more familiar with the session format rather than better at refactoring.

The author is one of three raters. One of the three labellers was the author, so author judgement may have affected the agreement results. To make this easier to inspect, §4.2.1 reports pairwise κ rather than only a single aggregate score. The other two labellers were later placed in the with-feedback group of the user study, but they completed all labels before taking part in any study sessions. Their labels therefore could not have been shaped by their later behaviour in the experiment. The remaining concern goes the other way: two of the three with-feedback participants had already seen the divergence-point taxonomy during labelling. For that reason, §4.3.1 reports per-participant trajectories rather than hiding this inside the group mean.

5.2.3 Generalisation and implementation scope

The injection sessions are controlled exercises. The 45 injection sessions in the primary test set follow a scripted playbook, with one specific bad behaviour added to each session. This makes them useful for checking whether the detector finds known divergence kinds, but they should not be treated as typical refactoring sessions. The precision and recall figures in §4.2.2 should therefore be read as controlled-test results, not as direct estimates for ordinary development work.

`SuboptimalOrdering` is based on the playbook author's judgement. The five ordering-injection sessions are labelled `SuboptimalOrdering` because the playbook author chose an order that seemed worse than the available alternatives. The score formula agrees with that judgement in only 1 of the 5 sessions (§4.2.2, session 022); the other 4 have zero or negative magnitude under the production formula. The *Ordering* precision and recall numbers should therefore be read with this mismatch in mind.

The agent extension is a feasibility experiment, not an agent evaluation. The eight-agent, 48-session extension tests how detectors designed for human IDE sessions behave on a different kind of actor. It is not an evaluation of the tool as an agent-monitoring framework. Three of the four detector kinds fit agent traces poorly (§4.3.2). Agents have no IDE refactoring surface, so *IDE-Replay* fires on every structural refactoring without offering a realistic alternative. The six-checkpoint commit-gap threshold is rarely reached within six sessions, and the 2-second edit-burst debouncer collapses each agent response into one burst. The agent result should therefore not be read as evidence that feedback had no effect on agent behaviour. It mainly shows that these detectors are not well matched to agent edit patterns.

The reorder enumerator has a hard size budget. The *Ordering* detector skips reorder windows with more than 7 refactoring steps, because enumerating all valid orderings becomes expensive beyond that point (§3.3.2). The recorded sessions rarely reach this limit. However, a session with a longer uninterrupted chain of refactorings could lose its largest windows from the *Ordering* analysis.

5.3 Headline findings

With those limitations in mind, the project supports four main findings.

First, the kind classifier performs well on the controlled-injection sessions. Against the inter-rater-agreed `expected_kinds` labels, per-kind precision is 1.00 for all four kinds. The agreement check is also strong: Cohen's $\kappa = 1.00$ for *Ordering* and *IDE-Replay* on all three rater pairs, and $\kappa = 0.72$–1.00 for *Rework* and *Hygiene* (§4.2.1, §4.2.2). Every detected injection with a positive-magnitude alternative trajectory ranked at top-1 within its session.

Second, the score formula is locally stable around its production weights. On the rankable user-study sessions (25 of 30), changing one weight at a time preserves the top-ranked divergence point in 96.5% of cases. Perturbing all weights at once lowers this to 84.6% (§4.1.1). This suggests that the formula is not fragile near the chosen weights, although the result is still local: larger or differently structured changes to the weights can change the rankings.

Third, the user-study group comparison points in a clear direction. Across the six sessions, participants who saw feedback gained +22.3 on the gain-stripped process score (mean S1 26.7 → S6 49.0) and reduced total divergence points from 12 to 2. The no-feedback participants lost 2.0 points (mean S1 22.5 → S6 20.5) and their total divergence points rose from 10 to 15. With $n = 3$ vs $n = 2$, the study is randomised but small, so no p-value is claimed. The result is reported only as a direction-of-effect signal. The two groups start at comparable totals in S1 and diverge from S3 onwards, which makes it less likely that the with-feedback improvement simply reflects the playbook getting easier over time (§4.3.1).

Fourth, the agent extension is best read as a feasibility experiment, not as a full evaluation of the tool on agents. It covers eight agents across model × harness combinations, with six feedback agents, two no-feedback baselines, and 48 sessions in total. The detectors

were designed around human IDE sessions, and they do not transfer cleanly to agent traces: of the 42 divergence points found across the agent sessions, 41 are *IDE-Replay*, because agents have no IDE refactoring surface to switch to and the other three detector kinds rarely fire (§4.3.2). The feedback condition also shows no clear effect on agent process scores. Both groups have positive slopes, but the no-feedback baseline slope is larger (+1.3 versus +0.6), so the visible difference runs in the opposite direction to the human result. The transcripts add useful context that the score does not capture: five of the six feedback agents explicitly write down plans for what they will change next, but the gain-stripped ΔJ still does not separate from the no-feedback group. This fits the detector-scope limitation, since most agent adjustments concern build-broken time and commit cadence, while the measured agent sessions are dominated by *IDE-Replay*.

5.4 Future work

The most useful next steps are the ones that would test the main findings on stronger evidence:

1. **Larger randomised human study.** The strongest evidence for a feedback effect is the with-feedback vs no-feedback group comparison in §4.3.1, but it is based on only five participants. A larger study would show whether the direction-of-effect signal holds with more people, and would make it possible to estimate between-participant variance properly.

2. **Adapting the tooling to agent traces.** The current detectors are built around human IDE behaviour, so they do not fit agent traces cleanly. A version aimed at agents would need different assumptions: grouping edits by tool call rather than by short edit bursts, tracking commits per agent response rather than through the six-checkpoint window, and checking which refactoring tools the agent can actually use. That would let the same scoring framework be tested on agents without the current detector mismatch.

3. **IDE refactoring tools for agents.** The clearest agent-side limitation is that feedback can recommend an IDE refactoring, but the agent has no way to invoke it. Exposing IntelliJ refactorings as agent tool calls would make that feedback actionable and would allow the *IDE-Replay* count to change across sessions.

5.5 Closing

This thesis shows that the path through a refactoring session can be made visible, not just inferred from the final code. The score and detector developed here are still limited by calibration, dataset size, and detector coverage, but the results suggest that concrete process feedback is feasible: controlled divergences can be recovered reliably, rankings remain stable near the chosen weights, and the user study points in an encouraging direction. The agent extension also shows where the same idea needs adapting for traces that do not look like human IDE work.

The main outcome is a practical framing for future tools. Refactoring trajectories can be compared with explicit alternatives, and the differences can be shown back to the person or system doing the work. With larger human studies, broader refactoring support, and agent-specific detectors, this creates a route towards tools that help developers not only end with cleaner code, but get there through a cleaner and steadier process.

Chapter 6

Declarations

6.1 Use of Generative AI

I acknowledge the use of Claude Code (Anthropic, https://claude.com/claude-code) and the underlying Claude Opus / Sonnet models while implementing the tool described in this thesis. AI assistance was particularly useful for these development tasks, under my direct supervision:

- *Repetitive adapter and wrapper code.* The analysis pipeline bridges IntelliJ's recorded refactoring events to Eclipse JDT's refactoring API. Many refactoring specs, such as `RenameMethod`, `RenameField`, `RenameClass`, `RenameLocalVariable`, `ExtractMethod`, `InlineMethod`, `ChangeMethodSignature`, and `ParameterizeVariable`, required similar wrappers with small differences in fields, preconditions, and follow-up validation. After writing the first versions and fixing the shared interface, validation logic, and AST-equivalence check, I used AI assistance to draft further wrappers following the same pattern.

- *Serialisation and test scaffolding.* AI assistance was used for Kotlin data classes for the analysis-report JSON schema and for fixture-builder code in the test suite.

- *Understanding unfamiliar APIs.* I used AI to help explain parts of the IntelliJ Platform plugin SDK, the Eclipse Equinox bootstrap, and the JCEF dashboard environment, especially where the official documentation was limited.

- *Fixture and playbook ideas.* AI was used to help generate ideas for the Java fixtures and scripted playbooks used in the manual-injection, user-study, and agent-comparison sessions. These were purely test data. I edited the scenarios manually, and verified that they exercised the intended behaviours.

All AI-assisted code was reviewed manually. The analysis, refactoring, and dashboard modules are also covered by tests, which provide a separate check on the behaviour of the implementation.

For the report itself, the content, argument, and findings are my own. Claude (Anthropic, https://claude.ai) was used for proofreading, structural feedback on draft text, generating an initial outline for Chapter 4 that I then revised and populated with my own analysis, and help with LaTeX formatting for tables and figures. No AI-generated content has been presented as my own independent work.

6.2 Ethical Considerations

The project includes a small user study with five human participants, referred to as P1–P5, and a separate agent comparison with no human subjects. All participants were senior MEng Computing students at Imperial College London. They consented to having their sessions recorded and analysed, on the basis that no personally identifying information would appear in the report or released materials. The participant traces and captured edits are pseudonymised, and the underlying recordings are kept under access controls in the project repository.

The project fits Imperial's description of low-risk research, as it involved only non-sensitive session data from consenting participants and posed no risk beyond ordinary development work [58]. No formal ethics approval was required. It did not use medical data, sensitive personal data, or third-party-owned datasets.

6.3 Sustainability

The project's compute footprint was kept small by the Phase A / Phase B pipeline split described in the methodology chapter. The expensive analysis work, including shadow-repository reconstruction, RefactoringMiner mining, and build/test execution at each checkpoint, is run once per recorded session and cached. Later scoring, sensitivity, ablation, and Monte Carlo experiments reuse this cached output and rerun only the cheaper scoring phase. This avoids repeating the full analysis for every weight configuration.

All experiments were run on a single local workstation. No GPU or cloud-compute resources were consumed.

6.4 Availability of Data and Materials

The project materials are available as follows:

1. **Source code**: https://github.com/EthanHosier/fyp. The repository contains the IntelliJ plugin, analysis pipeline, JCEF dashboard, and experiment drivers.
2. **Recorded sessions, fixtures, and supporting material**: the recorded sessions, playbook prompts, fixtures, and session-recording protocol are stored under `tool/fixtures/`. This includes the 45 hand-recorded injection sessions, 30 user-study sessions across 5 participants, and 48 agent-comparison sessions across 8 agent configurations. The directory layout is documented in `tool/fixtures/README.md`.
3. **Reproducibility of Chapter 4**: the numerical claims, tables, and plots in Chapter 4 can be reproduced with `tool/fixtures/notebooks/experiments.ipynb`. The notebook reads the per-session cached analysis files and produces the reported aggregates. Setup instructions are provided in `tool/fixtures/notebooks/README.md`.
4. **Participant data**: participant data is pseudonymised. No personally identifying information is included in the repository or in this report.

Appendix A

Appendix

This appendix is intentionally left available for supplementary material that is useful to include in the submitted report but too detailed for the main chapters.

Bibliography

[1] William F. Opdyke. *Refactoring Object-Oriented Frameworks*. PhD thesis, University of Illinois at Urbana-Champaign, Urbana-Champaign, IL, USA, 1992.

[2] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.

[3] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 2 edition, 2018.

[4] Tom Mens and Tom Tourwé. A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139, 2004.

[5] Emerson R. Murphy-Hill, Chris Parnin, and Andrew P. Black. How we refactor, and how we know it. In *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pages 287–297, 2009.

[6] Miryung Kim, Dongxiang Cai, and Sunghun Kim. An empirical investigation into the role of API-level refactorings during software evolution. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pages 151–160, 2011.

[7] Pouria Alikhanifard and Nikolaos Tsantalis. A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. *ACM Transactions on Software Engineering and Methodology*, 34(2), 2025.

[8] Jean-Rémy Falleri and Matias Martinez. Fine-grained, accurate, and scalable source code differencing. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, 2024.

[9] Dhruv Gautam, Spandan Garg, Jinu Jang, Neel Sundaresan, and Roshanak Zilouchian Moghaddam. RefactorBench: Evaluating stateful reasoning in language agents through code. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[10] Islem Bouzenia and Michael Pradel. Understanding software engineering agents: A study of thought-action-result trajectories. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.

[11] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah AlOmar. Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software*, 235:112762, 2025.

[12] Veselin Raychev, Max Schäfer, Manu Sridharan, and Martin Vechev. Refactoring with synthesis. In *Proceedings of the 2013 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 339–354. ACM, 2013.

[13] Yu Lin, Xin Peng, Yumin Cai, Danny Dig, Diwen Zheng, and Wenyun Zhao. Interactive and guided architectural refactoring with search-based recommendation. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 535–546. ACM, 2016.

[14] Mark Harman and Laurence Tratt. Pareto optimal search-based refactoring at the design level. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation (GECCO '07)*, pages 1106–1113. ACM, 2007.

[15] M. Mohan and Des Greer. Using a many-objective approach to investigate automated refactoring. *Information and Software Technology*, 112:83–101, 2019.

[16] Nikolaos Nikolaidis, Nikolaos Mittas, Apostolos Ampatzoglou, Daniel Feitosa, and Alexander Chatzigeorgiou. A metrics-based approach for selecting among various refactoring candidates. *Empirical Software Engineering*, 29(1), 2024.

[17] Matheus Paixão, Mark Harman, Yuanyuan Zhang, and Yijun Yu. An empirical study of cohesion and coupling: Balancing optimisation and disruption. *IEEE Transactions on Evolutionary Computation*, 22(3):394–414, 2017.

[18] Naouel Moha, Yann-Gaël Guéhéneuc, Laurence Duchien, and Anne-Françoise Le Meur. DECOR: A method for the specification and detection of code and design smells. *IEEE Transactions on Software Engineering*, 36(1):20–36, 2010.

[19] Thanis Paiva, Amanda Damasceno, Eduardo Figueiredo, and Cláudio Sant'Anna. On the evaluation of code smells and detection tools. *Journal of Software Engineering Research and Development*, 5, 2017.

[20] Francesca Arcelli Fontana, Jens Dietrich, Bartosz Walter, Aiko Yamashita, and Marco Zanoni. Antipattern and code smell false positives: Preliminary conceptualization and classification. In *Proceedings of the 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, volume 1, pages 609–613. IEEE, 2016.

[21] Simone Scalabrino, Mario Linares-Vásquez, Rocco Oliveto, and Denys Poshyvanyk. A comprehensive model for code readability. *Journal of Software: Evolution and Process*, 30(6):e1958, 2018.

[22] Danilo Silva and Marco Tulio Valente. Refdiff: Detecting refactorings in version histories. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, pages 269–279. IEEE Press, 2017.

[23] Vittorio Cortellessa, Daniele Di Pompeo, Vincenzo Stoico, and Michele Tucci. Many-objective optimization of non-functional attributes based on refactoring of software models. *Information and Software Technology*, 157:107159, 2023.

[24] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.

[25] Anna Maria Eilertsen and Gail C. Murphy. A study of refactorings during software change tasks. *Journal of Software: Evolution and Process*, 2024.

[26] Kostadin Damevski, David C. Shepherd, Johannes Schneider, and Lori L. Pollock. Mining sequences of developer interactions in visual studio for usage smells. *IEEE Transactions on Software Engineering*, 43(4):359–371, 2017.

[27] Wouter Poncin, Alexander Serebrenik, and Mark van den Brand. Process mining software repositories. In *Proceedings of the 15th European Conference on Software Maintenance and Reengineering (CSMR)*, pages 5–14. IEEE, 2011.

[28] Manaal Basha, Aimêe M. Ribeiro, Jeena Javahar, Cleidson R. B. de Souza, and Gema Rodríguez-Pérez. CodeWatcher: IDE telemetry data extraction tool for understanding coding interactions with LLMs. In *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution (ICSME), Tool Demonstration Track*, 2025.

[29] Aiko Yamashita, Fábio Petrillo, Foutse Khomh, and Yann-Gaël Guéhéneuc. Developer interaction traces backed by ide screen recordings from think-aloud sessions. In *Proceedings of the 15th International Conference on Mining Software Repositories (MSR) (Data Showcase)*, pages 50–53, Gothenburg, Sweden, 2018. ACM. Dataset paper providing IDE interaction traces with synchronized screen recordings.

[30] Diego Cedrim, Alessandro Garcia, Melina Mongiovi, Rohit Gheyi, Leonardo Sousa, Rafael de Mello, Baldoino Fonseca, Márcio Ribeiro, and Antonio Chávez. Understanding the impact of refactoring on smells: A longitudinal study of 23 software projects. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 465–475. ACM, 2017.

[31] Stas Negara, Nicholas Chen, Mohsen Vakilian, Ralph E. Johnson, and Danny Dig. A comparative study of manual and automated refactorings. In *Proceedings of the 27th European Conference on Object-Oriented Programming (ECOOP)*, volume 7920 of *LNCS*, pages 552–576. Springer, 2013.

[32] Mohsen Vakilian, Nicholas Chen, Stas Negara, Balaji Ambresh Rajkumar, Brian P. Bailey, and Ralph E. Johnson. Use, disuse, and misuse of automated refactorings. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, pages 233–243. IEEE, 2012.

[33] Ward Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, pages 29–30. ACM, 1992.

[34] Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. Technical debt: from metaphor to theory and practice. *IEEE Software*, 29(6):18–21, 2012.

[35] Yida Tao and Sunghun Kim. Partitioning composite code changes to facilitate code review. In *Proceedings of the 12th IEEE/ACM Working Conference on Mining Software Repositories (MSR)*, pages 180–190. IEEE, 2015.

[36] Marco Di Biase, Magiel Bruntink, Arie van Deursen, and Alberto Bacchelli. The effects of change decomposition on code review: A controlled experiment. *PeerJ Computer Science*, 5:e193, 2019.

[37] Kim Herzig and Andreas Zeller. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*, pages 121–130. IEEE, 2013.

[38] Gunnar Kudrjavets, Nachiappan Nagappan, and Ayushi Rastogi. Do small code changes merge faster? A multi-language empirical investigation. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. IEEE, 2022.

[39] G. Ann Campbell. Cognitive complexity: A new way of measuring understandability, 2018. SonarSource white paper, not peer-reviewed.

[40] Marvin Muñoz Barón, Marvin Wyrich, and Stefan Wagner. An empirical validation of cognitive complexity as a measure of source code understandability. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, 2020. Best Full Paper award.

[41] Luigi Lavazza, Abdallah Z. Abualkishik, Geng Liu, and Sandro Morasca. An empirical evaluation of the "Cognitive Complexity" measure as a predictor of code understandability. *Journal of Systems and Software*, 197:111561, 2023.

[42] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.

[43] Ilja Heitlager, Tobias Kuipers, and Joost Visser. A practical model for measuring maintainability. In *Proceedings of the 6th International Conference on the Quality of Information and Communications Technology (QUATIC)*, pages 30–39. IEEE, 2007.

[44] Raymond P. L. Buse and Westley R. Weimer. Learning a metric for code readability. *IEEE Transactions on Software Engineering*, 36(4):546–558, 2010.

[45] Radu Marinescu. Detection strategies: Metrics-based rules for detecting design flaws. In *Proceedings of the 20th IEEE International Conference on Software Maintenance (ICSM)*, pages 350–359. IEEE, 2004.

[46] James M. Bieman and Byung-Kyoo Kang. Cohesion and reuse in an object-oriented system. In *Proceedings of the 1995 Symposium on Software Reusability (SSR'95)*, pages 259–262. ACM Press, 1995. Introduces Tight Class Cohesion (TCC) metric.

[47] Stefan Wagner, Andreas Goeb, Lars Heinemann, Michael Kläs, Constanza Lampasona, Klaus Lochmann, Alois Mayr, Reinhold Plösch, Andreas Seidl, Jürgen Streit, and Adam Trendowicz. Operationalised product quality models and assessment: The Quamoco approach. *Information and Software Technology*, 62:101–123, 2015.

[48] Jagdish Bansiya and Carl G. Davis. A hierarchical model for object-oriented design quality assessment. *IEEE Transactions on Software Engineering*, 28(1):4–17, 2002.

[49] Don Coleman, Dan Ash, Bruce Lowther, and Paul Oman. Using metrics to evaluate software system maintainability. *Computer*, 27(8):44–49, 1994.

[50] Eclipse Foundation. JDT/FAQ. Eclipse Wiki, 2026. Consulted 2026-05-18.

[51] Maurice G. Kendall. *Rank Correlation Methods*. Charles Griffin & Co., London, 1948.

[52] Alan Agresti. *Analysis of Ordinal Categorical Data*. John Wiley & Sons, Hoboken, NJ, 2 edition, 2010.

[53] Michaela Saisana, Andrea Saltelli, and Stefano Tarantola. Uncertainty and sensitivity analysis techniques as tools for the quality assessment of composite indicators. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 168(2):307–323, 2005.

[54] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 4765–4774, 2017.

[55] Erik Štrumbelj and Igor Kononenko. Explaining prediction models and individual predictions with feature contributions. *Knowledge and Information Systems*, 41(3):647–665, 2014.

[56] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[57] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977.

[58] Imperial College London. Research ethics. https://www.imperial.ac.uk/admin-services/early-career-researcher-institute/learning-and-development/policies-and-guidance/research-ethics/, 2026. Accessed: 2026-05-28.